

Rapport de stage

Jorel Raphaël

18 août 2014

Résumé

Les compilateurs d'aujourd'hui permettent de faire beaucoup d'optimisations de programmes, grâce à de nombreuses options de compilations que l'on peut leur fournir. Les plus connues sur le compilateur GCC par exemple, sont -O0, -O1, -O2, -O3, qui sont différents niveaux d'optimisation plus ou moins efficaces. Mais il en existe bien d'autres, et il est à ce titre bien compliqué de savoir pour un programme donné, quelles seront les options (ou flags) qui donneront les meilleures performances (vitesses d'exécution, taille de code, ...).

C'est face à cette problématique que Gregory Fursin a eu l'idée de créer un outil qui permettrait d'automatiser cette tâche. Travaillant d'abord sur une version de cet outil s'appelant cTuning [1], il l'a totalement refondue depuis 2009 pour en créer une autre, nommée cMind[2]. Le but de ce document est de vous introduire le fonctionnement de l'outil afin que vous puissiez vous en servir.

Table des matières

1	Le contexte	3
1.1	Lieu	3
1.2	Objectifs	3
1.3	Un outil bien complexe	3
2	Expérience	3
2.1	Le projet	3
2.2	La commande générale	4
2.3	Installation	4
2.4	Configuration	4
2.5	Installation des jeux de données	5
2.6	Exploration des options de compilation	5
2.7	Graphiques	6
3	Explications	6
3.1	Fonctionnement général	6
3.1.1	Langage	6
3.1.2	Entrées	6
3.1.3	Dépôts	7
3.1.4	Modules	7
3.1.5	JSON	7
3.1.6	Ligne de commande	7
3.1.7	Alias	7
3.1.8	Modules externes	8
3.2	Utilisation	8
3.2.1	Ajout de données	8
3.2.2	Suppression / mise à jour	8
3.2.3	Configurer les descriptions	8
3.2.4	Chargement d'une description	9
3.3	Le projet	9

3.3.1	Lancement	9
3.3.2	Installation	9
3.3.3	Configuration	9
3.3.4	Jeux de données	12
3.3.5	Exploration	12
3.3.6	Résultats	14
4	Exemples	16
4.1	Visualisation	16
4.2	Lister les options explorées	17
4.3	Reproductibilité	17
4.4	Création de modules externes	17
4.4.1	Eléments de base	17
4.4.2	Accès aux données	18
4.4.3	Affichage	18
4.4.4	Comportement par défaut	19
4.4.5	Résumé	19
4.4.6	Noms de variables	19
5	Graphiques	19
5.1	Commentaires	29
A	Likwid	30
A.1	Présentation	30
A.2	Installation	30
A.3	Utilisation	30
A.4	Fiabilité	30
A.5	Instrumentation avec cMind	32
B	ICC	33
C	FAQ	33
C.1	Ajouter un nouvel algorithme	33
C.2	Ajouter un jeu de données	34
C.3	Faire un nouveau test (algorithme + jeux de données)	34
C.4	Ajouter une option de compilateur	34
C.5	Compiler avec une bibliothèque externe	34
C.6	Faire une exploration sans jeu de données	34

1 Le contexte

Je vais d'abord vous présenter le contexte de l'étude de cet outil, notamment l'endroit où je l'ai menée, ainsi que les objectifs que je devais atteindre.

1.1 Lieu

J'ai réalisé ce stage dans le laboratoire DALI de l'Université de Perpignan Via Domitia, université dans laquelle j'ai fait ma licence. Mon encadrant de stage fut M. Langlois, et j'ai été également en contact avec M. Parello et le doctorant Chems-Seddine Chohra, car leurs domaines de recherche concernent principalement l'optimisation de programmes.

1.2 Objectifs

L'objectif principal a été de comprendre le fonctionnement de cet outil cMind et s'en servir pour voir s'il était capable de trouver des combinaisons d'options de compilation intéressantes pour des algorithmes de sommation exacte de nombres flottants en double précision selon la norme IEEE-754, sur des vecteurs de tailles allant de 2^{14} à 2^{20} , de conditionnements allant de 10^8 à 10^{32} . Cela pose quelques contraintes au niveau des options, car certaines font des optimisations brutales sur les calculs et cela fausse les résultats. Les algorithmes étudiés sont :

- AccSum,
- FastAccSum,
- HybridSum,
- iFastSum,
- OnlineExact (avec TwoSum).

1.3 Un outil bien complexe

cMind m'a d'emblée paru être un outil dans lequel il était dur d'entrer. Cela tient sûrement plus à la documentation fournie qu'à mon expérience de chercheur. Le user guide donné en ligne a été la seule aide dont je me suis servi, mais malheureusement il n'est pas très clair. Finalement, la compréhension de l'outil est plus passée par de longues recherches dans les fichiers et dossiers de ce dernier que par les ressources de l'internet.

Je vais d'abord vous expliquer comment utiliser le projet que j'ai fait, puis je vous parlerai du fonctionnement de cMind afin que vous puissiez comprendre ce que j'ai écrit et pouvoir vous en servir par la suite.

2 Expérience

2.1 Le projet

Je joins avec ce rapport l'archive du projet, il suffira donc de le décompresser. Pour la suite, je considérerai que je me trouve à la racine du projet pour me référer aux différents fichiers utilisés au cours de ces explications.

La racine du projet contient plusieurs sous-répertoires et un exécutable.

- **data** - Jeux de données.
- **gen** - Dossier dans lequel sont mis tous les fichiers générés par les scripts et modules.
- **json** - Fichiers JSON de configurations.
- **likwid_test** - Tests pour vérifier la fiabilité de Likwid.
- **modules** - Modules externes pour cMind.
- **scripts** - Scripts automatisant mon expérience de l'outil.
- **sources** - Dossier contenant les fichiers sources du programme à tester.
- **misc** - Fichiers divers.

- **launch.sh** - Commande générale permettant de centraliser toutes les actions disponibles.

J'expliquerais le rôle de chacun des fichiers par la suite.

2.2 La commande générale

Pour permettre une utilisation plus simple, un fichier central permet de lancer toutes les actions, c'est un exécutable auquel il faut donner un paramètre permettant d'en lancer une en particulier. Ce fichier est `launch.sh`. Il définit deux variables d'environnement, `INSTALL_PATH` et `TOOL_DIR` qui servent, respectivement, à indiquer le chemin d'installation et le nom du dossier de l'outil cMind. Modifiez ces variables à votre convenance suivant l'endroit où vous souhaitez installer cMind.

Il y en a six options pour ce script (douze avec les options longues) :

- **-i, -install** - Installation de l'outil.
- **-c, -configure** - Configuration le projet pour les algorithmes que l'on étudie.
- **-d, -datasets** - Installation des jeux de données spécifiques à ces algorithmes.
- **-e, -explore-compiler-flags** - Explorations des options du compilateur pour ces algorithmes et ces jeux de données.
- **-g, -graphs** - Génération des graphiques à partir des mesures faites pendant les explorations.
- **-h, -help** - Afficher l'aide.

Ces différentes options indiquent au script général quels scripts plus spécialisés appeler.

2.3 Installation

Fichiers utilisés :

- `./launch.sh`
- `./scripts/install.sh`

Pour installer l'outil, il faut faire

```
$ ./launch.sh -i
```

Le script `./scripts/install.sh` est appelé.

Celui-ci se charge de créer le dossier et de télécharger les données de l'outil dans le dossier défini par les deux variables d'environnement de `./launch.sh`. Il vérifie tout d'abord que `svn` est installé, car ce dernier est indispensable pour le téléchargement. S'il ne l'est pas, il propose de l'installer pour pouvoir poursuivre. Si vous voulez installer cMind dans un dossier réservé au super-utilisateur, il faudra avoir ses droits, et rajouter l'option `-root` à la ligne de commande (après l'option `-i` absolument). Après avoir téléchargé l'outil, sa configuration est lancée. Celle-ci permet d'adapter cMind à son ordinateur (système d'exploitation, processeur, etc...) ainsi que de s'enregistrer sur le site du développeur pour l'utilisation en ligne de l'outil. Il est proposé aussi de télécharger des données relatives à son environnement de travail. Lors de la configuration, il faut prêter attention à deux points : le premier concerne "fordid writting", répondez "no" à cet endroit-là, sinon on ne pourra rien configurer ni installer dans cMind, le second concerne le système d'exploitation. Pensez à renseigner correctement votre système d'exploitation, cMind vous donne une liste de systèmes disponibles, il vous sera demandé si vous voulez vous enregistrer sur <http://cmind.org>, je vous conseille de le faire. Pour le reste, les options par défaut suffisent largement. Il y aura peut-être une étape un peu longue si vous choisissez de télécharger les données de cMind en ligne. Ici, je vous conseille de faire le téléchargement minimal. Pour finir, le script précise qu'il faut définir deux variables d'environnement avant l'utilisation de cMind.

2.4 Configuration

Fichiers utilisés :

- `./launch.sh`
- `./scripts/configure.sh`
- `./json/code.json`

- ./json/compiler.json
- ./modules/cme.install.py

Pour lancer la configuration, il faut faire

```
$ ./launch.sh -c
```

Le script ./scripts/configure.sh est appelé.

Dans un premier temps, ce dernier définit une classe de jeux de données, à laquelle seront rattachés ceux que nous utiliserons pour les algorithmes de sommation. Cette classe n'est pas nécessaire au bon fonctionnement, mais elle permet de définir un caractère commun entre les jeux de données.

Ensuite, le script ajoute une nouvelle description du compilateur GCC dont nous allons nous servir, ainsi que les codes sources. Pour cela, il utilise les deux fichiers JSON ./json/compiler.json et ./json/code.json.

Pour finir, un module externe est utilisé (./modules/cme.install.py) afin de configurer GCC pour votre machine s'il ne l'est pas. Si un message d'erreur apparaît à cette étape, c'est que GCC était déjà correctement configuré.

2.5 Installation des jeux de données

Fichiers utilisés :

- ./launch.sh
- ./scripts/datasets.sh
- ./scripts/gen_dataset_configuration.py
- ./json/dataset.json
- ./data/*.dat

Fichier généré et utilisé :

- ./gen/dataset.json

Pour installer et configurer les jeux de données qui seront utiles aux algorithmes de sommation, il faut faire

```
$ ./launch.sh -d
```

Le script ./scripts/datasets.sh est appelé.

Ce dernier parcourt le dossier ./data contenant les jeux de données et, sur chacun d'eux, appelle le script ./scripts/gen_dataset_*.py afin de les configurer pour cMind.

Le script en Python permet de déterminer la taille de chaque vecteur à sommer ainsi que leur conditionnement, il se sert du fichier modèle ./json/dataset.json, et ne rajoute que ces deux informations, puis sauvegarde le fichier dans ./gen/dataset.json. C'est ce dernier fichier qui est utilisé pour la configuration des jeux de données dans cMind, car les deux informations les concernant seront ensuite utilisées pour tracer les graphiques.

2.6 Exploration des options de compilation

Fichiers utilisés :

- ./launch.sh
- ./scripts/explore.sh
- ./scripts/gen_run_configuration.py
- ./modules/cme.explore_compiler_flags.py
- ./json/explore_compiler_flags.json

Fichier généré et utilisé :

- ./gen/explore_compiler_flags.json

Pour lancer l'exploration des options de compilation, il faut faire

```
$ ./launch.sh -e
```

Le script `./scripts/explore.sh` est appelé.

Cette opération est très longue suivant le nombre de combinaisons d'options que nous voulons tester sur les codes sources par exploration que nous souhaitons faire. J'ai fixé ce nombre à 5 pour ne pas prendre trop de temps. Le script `./scripts/gen_run_*.py` permet de générer automatiquement le fichier de configuration d'une exploration, il charge d'abord `./json/explore_*.json`, rajoute certaines valeurs, comme notamment la commande à lancer, le jeu de données à utiliser, l'endroit où sauvegarder les résultats des explorations, etc... il sauvegarde cela dans `./gen/explore_*.json`, fichier qui sera utilisé donc ensuite pour lancer une exploration.

Les explorations nécessitent un module externe, car c'est lui qui se charge de faire la sélection des options de compilation et d'organiser les appels aux modules internes de cMind. Ce module, écrit par le concepteur de l'outil, est le fichier `./modules/cme.explore_*.py`.

2.7 Graphiques

Fichiers utilisés :

- `./launch.sh`
- `./scripts/graphs.sh`
- `./modules/statistics.py`
- `./json/statistics.json`

Fichiers générés :

- `./gen/stats/run[23] ?/*.gnuplot`
- `./gen/stats/run[23] ?/*.dat`
- `./gen/stats/run[23] ?/*.eps`

Pour générer les fichiers de données et les graphiques, il faut faire

```
$ ./launch.sh -g
```

Le script `./scripts/graphs.sh` est appelé.

Afin de récupérer les données des compilation précédemment faites, un autre module externe est utilisé, `./modules/statistics.py`. Il utilise le fichier `./json/statistics.json` et se charge de créer des fichiers à partir de ces données ainsi que des scripts Gnuplot pour tracer les graphiques.

Les fichiers générés sont déplacés dans le dossier `./gen`, puis Gnuplot est appelé sur chaque script lui correspondant. Les graphiques se trouvent dans les dossiers `./gen/stats/run[23] ?/` au format EPS après l'opération.

3 Explications

Après avoir fait tout cela, vous devriez avoir quelques résultats, maintenant passons aux explications. Commençons par le fonctionnement de cMind.

3.1 Fonctionnement général

3.1.1 Langage

Cet outil est écrit en Python. La raison que je présume est que ce langage est portable, souple, puissant, fortement typé, mais aussi qu'il convient parfaitement à la manipulation de fichiers JSON, et cMind en utilise beaucoup pour décrire les données qu'il récupère, ainsi que toutes celles qu'il a déjà à l'installation.

3.1.2 Entrées

Les entrées dans cMind sont diverses. Il peut s'agir d'un compilateur, de codes sources, de packages, de jeux de données, etc... De ce fait, une entrée peut représenter plusieurs fichiers (codes sources), un seul (jeu de donnée), ou même aucun, car une entrée peut n'être qu'une description d'une entité déjà existante sur votre ordinateur.

3.1.3 Dépôts

cMind utilise des dépôts, dans lesquels il stocke les entrées. Les dépôts principalement utilisés sont :

- `default` - dépôt utilisé par défaut lors des manipulations de cMind.
- `ctuning-shared` - contient entre autres les exemples que cMind utilise en démonstration de l'outil.
- `ctuning` - la plupart des codes sources se trouvent là, ainsi que pas mal de packages externes.

Nous utiliserons seulement le dépôt par défaut par la suite.

3.1.4 Modules

cMind est composé de différents modules, chacun ayant ses actions spécifiques et d'autres communes à tous. Pour chaque module, un dossier du même nom peut être présent dans chaque dépôt de cMind. Ceci est dû au fait que les modules permettent de gérer les dépôts de cMind, chacun s'occupant des dossiers à son nom dans les dépôts.

3.1.5 JSON

cMind utilise des fichiers JSON pour décrire les entrées qu'il stocke. Toutes les entrées ont un fichier à ce format, nommé `data.json`. Mais l'outil se sert également de ce format de fichier pour communiquer au sein de lui-même, nous pouvons notamment en utiliser pour se servir des fonctions des modules.

3.1.6 Ligne de commande

cMind s'utilise en ligne de commande, il utilise la syntaxe

```
$ cm nom_module action
```

Les actions les plus simples sont `help`, listant toutes les actions possibles d'un module plus des descriptions sommaires de celles-ci. `info` permet de savoir le rôle d'un module, mais ce n'est pas ça qui vous fera mieux comprendre à quoi il sert, vu le peu d'informations donné.

La plupart du temps, il faut également fournir des paramètres (couples clé/valeur) à l'action. Deux manières sont possibles, la première étant de les donner sur la ligne de commande, tel

```
$ cm nom_module action cle1 = valeur1 ... cle_n = valeur_n
```

la seconde utilise un fichier JSON

```
{  
  "cle_1": "valeur_1",  
  .  
  .  
  .  
  "cle_n": "valeur_n"  
}
```

On l'utilisera de la manière qui suit, en supposant que le fichier s'appelle `fichier.json` :

```
$ cm nom_module action @fichier.json
```

Vous pouvez donner plusieurs fichiers JSON sur la ligne de commande, toujours précédés de `@`. L'avantage de cette seconde manière de procéder est qu'il est possible de donner des tableaux, indexés et associatifs, comme valeurs des paramètres.

3.1.7 Alias

Les alias permettent à cMind de lier un UID à une chaîne de caractères. cMind manipule bien mieux les valeurs hexadécimales je suppose, mais utiliser des alias permet de rendre les entrées plus lisibles pour nous et facilite leur manipulation. Chaque entrée a un UID et peut avoir un alias. Vous verrez souvent dans les fichiers JSON, le terme `_uoa` à la fin des noms de beaucoup de clés de paramètres. Ceci est l'acronyme de `Uid Or Alias`, donc vous pourrez mettre dans ces paramètres la forme que vous désirez lorsque vous voudrez désigner une entrée à traiter, bien que cela ne marche pas à tous les coups. Essayez

```
$ cm package installed prune_by_class_uoa=72f81bd39f84ed89
```

```
$ cm package installed prune_by_class_uoa=compiler
```

Vous devriez obtenir au moins un résultat dans le premier cas, et normalement aucun dans le second, alors que le nombre hexadécimal est l'UID de l'alias compiler.

3.1.8 Modules externes

Les modules externes sont une fonctionnalité très intéressantes de cMind. Cela permet d'utiliser l'outil et d'étendre ses possibilités de façon simple, en bénéficiant de son fonctionnement interne. Les modules externes s'utilisent exactement comme les modules internes, nous pouvons leur passer des fichiers JSON, cMind se chargera de le convertir en données exploitables et ils auront accès aux données, et donc aux entrées, que l'outil stocke.

3.2 Utilisation

3.2.1 Ajout de données

Maintenant, voyons une utilisation basique des dépôts avec l'ajout d'entrées. Supposons que vous vouliez ajouter un compilateur. Pour ce faire, tapez

```
$ cm ctuning.compiler add
```

Maintenant si vous vous rendez dans le répertoire où vous avez installé cMind, puis dans le dossier repos/, vous verrez tous les dépôts qu'utilise cMind. Allez dans celui nommé default, puis entrez dans le dossier .cmr. Ce nom est l'acronyme de CMind Repository, chaque dépôt a son dossier .cmr. Maintenant, allez dans le dossier ctuning.compiler, vous constaterez qu'il y a un dossier portant un nombre en hexadécimal. Si vous retapez la commande, un nouveau apparaîtra. Pour donner un nom au compilateur à ajouter, il faudra le renseigner sur la ligne de commande, ou dans un fichier JSON. Tapez, par exemple

```
$ cm ctuning.compiler add cm_data_uoa=mon-gcc
```

A présent, dans le dossier ctuning.compiler se trouve un nouveau sous-dossier du nom mon-gcc. cMind vous informe dans le terminal du chemin où se trouve la nouvelle entrée après l'ajout. Dans le dossier mon-gcc/.cm se trouve un fichier data.json. Il contient les informations générées par cMind pour n'importe quelle nouvelle entrée dans les dépôts, quelque soit le module utilisé.

Une entrée est donc représentée par un dossier dans cMind contenant un fichier data.json et éventuellement des fichiers supplémentaires suivant les cas (les codes source par exemple). Ce dossier est lui-même dans un autre correspondant au module utilisé pour la gestion de l'entrée.

Ceci dit, les informations générées automatiquement par cMind ne nous avancent pas énormément, mais il est possible d'en mettre qui décrivent plus précisément les entrées, et cela sans avoir à toucher les fichiers data.json. Pour cela, nous allons utiliser un fichier JSON, je le nommerais fichier.json. Créez-le et mettez dedans simplement :

```
{  
  "cm_data_uoa": "mon-gcc"  
}
```

Si vous tapez maintenant

```
$ cm ctuning.compiler add @fichier.json
```

vous obtiendrez une erreur, puisque ce compilateur existe déjà.

3.2.2 Suppression / mise à jour

Voici deux autres actions principales que vous utiliserez pour gérer les dépôts : delete (suppression) et update (mise à jour). Vous devez absolument fournir le paramètre de clé cm_data_uoa afin que cMind sache ce qu'il doit supprimer ou mettre à jour.

3.2.3 Configurer les descriptions

A présent, rajoutons des informations. Vous pouvez déjà spécifier un autre dépôt que default, grâce à la clé cm_repo_uoa. Ensuite, vous pouvez dire à cMind le nom de l'entrée qu'il doit afficher

quand il s'en sert (pour peu qu'il l'affiche). Pour ce faire, il faut préciser la clé `cm_display_as_alias`. Rajoutez-la avec n'importe quelle valeur dans `fichier.json` et faite

```
$ cm ctuning.compiler update @fichier.json
```

Dans le `data.json` de `mon-gcc`, la clé dont je viens de parler se trouvera à présent ajoutée, avec la valeur que vous avez donnée. Il serait donc logique de penser que toute paire de clé/valeur que l'on met dans le JSON passé en paramètre est rajoutée dans le fichier de description `data.json`... eh bien non, cela ne fonctionne qu'avec certains paramètres. Afin d'ajouter des informations comme bon vous semble, il faudra les mettre d'abord dans un tableau associatif, de clé `cm_array`, de la manière suivante :

```
{
  "cm_data_uoa": "mon-gcc",
  "cm_array": {
    "cle1": "valeur1",
    "cle2": "valeur2"
    ...
  }
}
```

Vous pouvez mettre tout ce que vous voulez, y compris des tableaux indexés et associatifs, mais il faudra respecter évidemment la syntaxe JSON, les tableaux indexés étant entourés de `[ ]` et ceux associatifs de `{ }`.

3.2.4 Chargement d'une description

Chaque entrée dans les dépôts est décrite par son fichier `data.json`, ce sont ces fichiers dont cMind se sert pour reconnaître ses entrées, et ainsi les utiliser selon leur configuration. Mais nous pouvons accéder au contenu de ces fichiers en ligne de commande, tapez par exemple

```
$ cm ctuning.compiler load mon-gcc
```

vous obtiendrez toutes les informations du compilateur que vous avez précédemment données à cMind.

3.3 Le projet

À présent, je vais parler un peu de ce que j'ai fait, et donc des scripts que j'ai écrits. Je n'expliquerais pas ligne par ligne ces derniers, je suppose que vous savez lire des scripts bash.

3.3.1 Lancement

Le script `launch.sh` n'est qu'une redirection vers les scripts spécialisés dans certaines tâches. Il permet l'affichage de l'aide également et définit les deux variables d'environnement qui précisent à quel endroit cMind sera installé et utilisé. Tous les scripts appelés vérifient que ces deux variables sont bien définies.

3.3.2 Installation

Je tenais juste à préciser que cMind considère l'architecture de votre processeur en `x86_64`, cela ne posera pas problème si elle est en `x86` car cMind n'utilise pas les options liées à l'architecture.

3.3.3 Configuration

Classes cMind permet de définir des classes. Cela permet notamment de reconnaître les entrées par "paquets" plutôt que individuellement. J'espérais pouvoir utiliser une classe et y affecter mes jeux de données pour dire à cMind de travailler dessus de manière automatique. Je n'ai même réussi à les afficher par classe dans la console, ceci est donc une fonctionnalité à approfondir.

Compilateurs L'ajout d'une description de compilateur permet principalement de définir ses options de compilation. Il faut décrire chaque option avec des paramètres qu'utilise cMind. La description de mon compilateur est en fait celle de GCC-4.6.x de trouvant dans le dépôt `ctuning-shared` à laquelle j'ai retiré les options `-Ofast` et `-funsafe-math`, car ces deux-là ne permettent pas d'avoir les résultats exacts des sommations de nombres flottants, à cause de leurs optimisations trop agressives sur les calculs. Cette description se trouve dans le fichier `./json/compiler.json`, vous remarquerez que la liste des options se trouve à la clé `cm_array`, conformément à ce que j'ai déjà dit. Je nomme ce compilateur `gcc-modified`.

Codes sources Les codes sources contiennent les algorithmes que nous avons à tester. Ils doivent également être décrit. Ces entrées-là ne se contentent pas d'une description, il leur faut aussi les fichiers à compiler. J'ai trouvé une solution fort compliqué pour ajouter la description et les codes sources via cMind, mais cela ne marche pas à tous les coups. Cette démarche est décrite dans le fichier `./misc/add/script.sh`. Mais ici, je déplace les fichiers sans passer par cMind. Afin de définir correctement des codes sources, il va falloir donner des informations précises à cMind, que je décris en dessous.

```
{
  "add_target_extension": {      # A déterminer, sert peut-être à Windows
    "dynamic": "exe",
    "static": "exe"
  },
  "build_scripts_names": {      # Informations sur les scripts de compilation,
    facultatif.
    "dynamic": {
      "build_dynamic_bin_c": "Build dynamic binary directly",
      "build_dynamic_bin_c_through_asm": "Build dynamic binary from C sources through
        assembler",
      "build_dynamic_bin_c_through_bc": "Build dynamic binary from C sources through
        LLVM BC",
      "build_dynamic_bin_c_through_link": "Build dynamic binary from C sources
        through object files and linking"
    },
    "static": {
      "build_static_bin_c": "Build static binary directly",
      "build_static_bin_c_through_asm": "Build static binary from C sources through
        assembler",
      "build_static_bin_c_through_bc": "Build static binary from C sources through
        LLVM BC",
      "build_static_bin_c_through_link": "Build static binary from C sources through
        object files and linking"
    }
  },
  "build_scripts_uoa": [        # Nom du dossier contenant les scripts de compilation.
    "build-generic"
  ],
  "clean_scripts_names": {      # Script de nettoyage avant chaque compilation (
    toujours la même valeur).
    "clean": "Universal cleaning"
  },
  "main_language": "c",         # Facultatif.
  "run_cmds": {                 # Noms de commandes à lancer.
    "default": {                 # Une commande.
      "dataset_classes_uoa": [    # La classe des jeux de données.
        "dataset.dat"
      ],
      "hot_functions": [],        # Les fonctions que l'on veut considérer particuliè-
        rement dans les mesures.
      "run_time": {               # Information pour chaque exécution.
        "run_cmd_main": "-c -a $$dataset_path$$$os_dir_separator$$$dataset_filename
          #$", # Argument de la ligne de commande.
        "run_cmd_out1": "tmp-output1.tmp", # Fichier pour la sortie standard.

```

```

    "run_cmd_out2": "tmp-output2.tmp",      # Fichier pour la sortie d'erreur
        standard.
    "run_correctness_output_files": [],      # A déterminer.
    "run_output_files": []                  # Fichiers sauvegardés après chaque
        exécution.
}
    "build_set_env": {
        "CM_LD_FLAGS_MISC": "-lpapi",      # Flags externes.
        "CM_CC": "icc"                     # Mauvaise façon de modifier le compilateur.
    }
},
"run_vars": {
    "CT_REPEAT_MAIN": "1"                  # Variable d'environnement à utiliser dans les codes
        sources.
},
"run_vars_desc": {                        # Sa description est toujours la même, je n'y ai pas
    touché.
    "##CT_REPEAT_MAIN": {
        "default_value": "1",
        "desc_text": "Main function/kernel repetition",
        "explorable": "yes",
        "explore_level": "1",
        "explore_start": "0",              # Valeur minimale.
        "explore_step": "10",
        "explore_stop": "1000000",        # Valeur maximale.
        "field_size": "7",
        "forbid_disable_at_random": "yes",
        "property": "yes",
        "skip_form_refresh": "yes",
        "sort_index": "1",
        "type": "integer"
    }
},
"source_files": [                        # Fichiers sources.
    "main.c",
    "functions.c"
],
"target_file": "a"                       # Nom de l'exécutable.
}

```

Les paramètres La plupart des paramètres sont facilement compréhensibles avec une petite description, d'autres beaucoup moins. Il est nécessaire dans cette description de préciser les fichiers sources à utiliser, ainsi que le fichier cible. La première clé est obligatoire mais n'a pas l'air d'avoir d'effet sous Linux. Les noms de scripts ne sont que des informations utiles à l'utilisateur concernant les scripts permettant de compiler les fichiers sources. Ces derniers se trouvent dans le dossier `os.script/build-generic`, dans le dépôt default. On retrouve ici la classe des scripts que l'on doit donner, `build-generic`. Il y a d'autres dossiers dans `os.script` contenant des scripts diverses, mais ceux-là ne nous intéressent pas ici. Le script de nettoyage est toujours le même.

run_cmds Cette clé mérite une description plus particulière. Sa valeur est une liste de commandes. Chaque commande est référencée par un nom, qui sera la clé d'un tableau associatif. Les classes de jeux de données associées à chaque commande permettent de cibler le type de ces derniers que la commande transmettra à l'exécutable. Étrangement, ce paramètre n'est pas restrictif, il est tout à fait possible de passer un jeu de données ne faisant partie d'aucune classe précisée par la commande. Par contre, si l'on n'en passe aucun, `cMind` nous donnera la liste des jeux de données rattachée aux classes de la commande.

Il faut également préciser des paramètres d'exécution, comme la ligne de commande qui sera exécutée, cela dépendra de votre programme. Il peut être intéressant de récupérer la sortie standard et la sortie d'erreur pour déboguer un problème inattendu d'un programme. Le dernier point important est l'expression `#{dataset_path}###{os_dir_separator}###{dataset_filename}#`. Grâce à elle,

cMind gère le passage en ligne de commande du jeu de donnée que nous souhaitons utiliser. Cela va sans dire que le programme testé doit récupérer le jeu de données lui-même, ce n'est pas cMind qui le fait pour lui.

Pour finir, la clé `build_set_env` permet de redéfinir toutes les variables d'environnement utilisées par le script de compilation. Cela permet en outre de faire l'édition de liens avec des bibliothèques externes.

CT_REPEAT_MAIN Cette variable est définie par cMind comme variable d'environnement lors des exécutions. Il faudra alors modifier vos codes en l'utilisant afin de répéter la partie du programme qui vous intéresse lors des mesures. cMind exécutera votre programme avec différentes valeurs de `CT_REPEAT_MAIN` jusqu'à, je suppose, trouver celle qui est la plus intéressante. Dans le cas où les mesures de temps sont trop faibles, si vous ne modifiez pas votre code, cMind renverra des temps inexistantes. Vous pourrez par la suite lui dire de ne pas faire fluctuer la valeur de `CT_REPEAT_MAIN`, et de la laisser à celle par défaut.

Si vous ouvrez le fichier `./sources/main.c`, vous trouverez la façon d'utiliser cette variable dans un code. Il peut paraître étonnant de devoir instrumenter soi-même son programme, mais c'est ainsi que cela fonctionne.

3.3.4 Jeux de données

Les jeux de données (datasets) font l'objet d'une installation à part, pour permettre d'en rajouter par la suite sans que toutes les configurations précédentes soient refaites. Chacun d'eux est d'abord analysé à partir de son nom par un script Python pour déterminer la taille du vecteur de nombres flottants qu'il contient, ainsi que son conditionnement. Ainsi, les jeux de données se voient attribuer deux valeurs qui permettront de les différencier. Ils sont également tous rattachés à la même classe. Evidemment, cela concerne mes jeux de données, vous pouvez écrire un fichier JSON statique correspondant à un seul jeu de données que vous avez créé. Voici un fichier de configuration de jeu de données simple.

```
{
  "cm_data_uoa": "truc", # Nom du jeu de données.
  "cm_description": "Dataset for summing", # Pour l'affichage.
  "cm_display_as_alias": "",
  "cm_array": {          # Informations complémentaires.
    "cm_classes_uoa": [
      "62a455f8c8042f90", # Classe générale "dataset".
      "dataset.dat"       # Classe personnelle, pour l'exemple.
    ],
    "cm_properties": {    # Informations personnelles.
      "size": "1024",
      "cond": "32"
    },
    "dataset_files": [    # Fichiers qui contiennent les données physiquement.
      "data.dat"
    ]
  }
}
```

N'oubliez pas que vous pouvez mettre tout ce que vous voulez dans le paramètre `cm_array`, je ne donne donc qu'un exemple. Le(s) fichier(s) à préciser dans `dataset_files` permet de dire à cMind où se trouvent les valeurs du jeu de données. Il doit être dans le dossier du jeu de données.

3.3.5 Exploration

Une exploration permet de choisir des options de compilation, afin d'en créer des combinaisons qui seront appliquées à une compilation des codes sources. Ici, les explorations utilisent le module externe écrit par le concepteur de cMind, les combinaisons sont faites de façon pseudo-aléatoire.

Configuration Afin de lancer une exploration, il est nécessaire de passer par un fichier de configuration. Voici un descriptif des paramètres à renseigner.

```
{
  "prepare_state": {
    "tuning_repo_uoa": "default",          # Dépôt à utiliser.
    "compiler_package_name": "gcc",        # Package du compilateur.
    "compiler_fuzzy_match": "yes",         # Complète automatiquement les noms incomplets
                                          de package.
    "prune_program_by_class": "dc6b904ae894e1cb", # Valeur par défaut.
    "program_uoa": "samples",              # Codes sources à utiliser.
    "run_cmd_key": "default",              # Commande à lancer.
    "dataset_uoa": "text.txt",             # Jeu de données à utiliser.
    "compilation_type": "dynamic",         # Type de compilation.
    "compile_script": "build_dynamic_bin_c", # Script de compilation utilisé.
    "force_compile_flags": "-O3",          # Option par défaut.
    "skip_compile_output_redirect": "yes",  # A déterminer.
    "use_perf": "no",                      # A déterminer
    "repeat_for_statistics": "2",          # Nombre de répétitions des mesures sur
                                          une combinaison d'options.
    "statistics_module": "ctuning.filter.variation", # Module utilisé pour les
                                          mesures.
    "only_prepare_state": "no",            # A déterminer.
    "update_state": {
      "keep_all_files": "yes"              # A déterminer.
    },
    "suppress_output_when_running_pipeline": "yes", # Désactive l'affichage des
                                          informations de certain module.
    "output_state_after_running_pipeline": "no",    # A déterminer.
    "record_state_before_running_pipeline_to_file": "tmp_pipeline_input.json", #
                                          Sauvegarde de l'état du pipeline avant l'exploration.
    "record_state_after_running_pipeline_to_file": "tmp_pipeline_output.json", #
                                          Pareil mais après.
    "output_known_characteristics": "yes",          # A déterminer.
    "save_results_to_ctuning_space_uoa": "samples" # Dossier ou l'on sauvegarde les
                                          mesures dans ctuning.space.
  },

  "ctuning_compiler_uoa": "gcc-modified",          # Description du compilateur à utiliser (
                                          options).
  "number_of_iterations": "50",                   # Nombre de combinaisons à faire.
  "check_base_flags": "no",                        # N'utiliser que les options de base (-O0
                                          , -O1, -O2, -O3, -Os).
  "remove_base_flags_from_combination": "yes",     # Supprimer les options de base des
                                          combinaisons d'options.
  "calibrate_kernel": "yes",                       # Faire varier CT_REPEAT_MAIN.
  "check_output": "yes",                           # Vérifier la sortie (il faut maintenant
                                          comprendre comment s'en servir).
  "random_seed_number": "12340"                    # Graine permettant de reproduire les
                                          combinaisons d'options d'une exploration sur l'autre.
}
```

On précise à cMind les codes sources à utiliser, la commande des codes sources (reliée au paramètre de clé `run_cmds` de ces derniers), le jeu de données à utiliser (transmis via l'expression vue plus haut), la description du compilateur et de ses options, etc... Cependant, la description du compilateur ne dit pas à cMind comment l'utiliser, c'est pourquoi il faut lui préciser avec le package du compilateur à utiliser. Typiquement, cela permettra de dire à cMind qu'il faut se servir de la commande `gcc` pour compiler un programme, que `-o` permet de spécifier la sortie, etc... Concernant la commande, si elle n'est pas définie dans la description des codes sources, alors ce sera la dernière définie qui sera choisie (à moins qu'il n'y ait aucune commande auquel cas, cMind exécute le programme sans paramètres ni redirection de sortie). Par contre, si l'une des commandes se nomme `default`, elle sera choisie par défaut lorsque celle que vous demanderez n'existe pas.

Nous pouvons également dire à cMind de n'utiliser que la valeur par défaut de `CT_REPEAT_MAIN` grâce à `calibrate_kernel`, ce qui peut être très utile pour ne pas avoir à remanier son code

source si nous ne voulons plus utiliser la variable dans le code. Comme ce module sert à explorer des combinaisons d'options, il est évidemment possible de préciser le nombre de combinaisons à tester.

Lancement Afin d'utiliser ce module de façon automatique avec tous les algorithmes que j'avais à tester sur tous les jeux de données, j'ai fait script bash et un script Python. Le script Python permet d'adapter le fichier de configuration d'exploration aux jeux de données et algorithmes, le script bash liste ces derniers, appelle le script Python et lance les explorations.

A propos du module Au début, lorsque je commençais à utiliser cMind avec les exemples, j'avoue avoir été très étonné que l'exploration ne fasse pas partie intégrante de l'outil. Ce module externe gère tout ce qui concerne l'exploration des options. C'est lui qui les choisit aléatoirement, et les valeurs de celles-ci si elles en demandent. Tout le reste est délégué aux fonctionnalités de l'outil, comme par exemple le chargement des données.

3.3.6 Résultats

Les résultats d'une exploration sont stockés dans le dossier `ctuning.space`, dans un dossier du nom que nous avons donné lors de la configuration d'exploration. Ces résultats se retrouvent dans le fichier `data.json` du dossier.

Espaces Tiré du nom du module, je nomme ainsi l'ensemble des résultats d'une ou plusieurs explorations stocké dans le même dossier, et donc le même fichier `data.json`. Cela est possible car cMind n'efface pas un espace lorsqu'on souhaite y rajouter des résultats d'une nouvelle exploration.

Points Un point désigne toutes les informations rattachées à une combinaison d'options de compilation. Elles sont très nombreuses, celles qui nous intéressent concernent les temps d'exécution et le jeu de données utilisés pour chacun, ainsi que la combinaison d'options utilisée.

Module de tracé Afin de pouvoir utiliser ces données, j'ai écrit un module externe. J'aurais pu utiliser le module de tracé intégré (`ctuning.space`), mais celui-ci utilise `matplotlib` sous Python, et je n'ai pas cherché à savoir comment l'utiliser plus profondément. De plus, les propriétés des jeux de données ne sont pas renseignées dans les points des espaces, alors que j'en avais besoin. Le module ne fait que récupérer les données relatives aux points, les organise et crée des scripts Gnuplot pour faire les tracés avec ce dernier.

Je vais donner une description complète de ce module, afin qu'il puisse être réutilisé, surtout que je l'ai écrit pour qu'il puisse s'adapter aux propriétés des jeux de données que nous utilisons.

Configuration Pour rester dans l'esprit de cMind, et permettre une utilisation simple, j'ai utilisé un fichier JSON. Voici un descriptif.

```
{
  "dir_output": "gen",      # Répertoire où seront mis les fichiers générés ( "." par défaut ).
  "cm_spaces": [           # Les espaces à tracer se trouvant dans ctuning.space.
    "acc_sum",
    "fast_acc_sum",
    "hybrid_sum",
    "ifast_sum",
    "online_exact_two_sum"
  ],
  "datasets_properties": [ # Liste des propriétés des jeux de données.
    "size",
    "cond"
  ]
  "graphs": [             # Les différents graphiques à tracer.
    {
      "name": "size",      # Nom du graphique.
    }
  ]
}
```

```

    "axis": [          # Axes.
        {
            "name": "n",          # Axe x.
            "value": "size",
            "logscale": "yes"     # Echelle logarithmique (facultatif)
        },
        {
            "name": "Temps",      # Axe y.
            "value": "global_run_kernel_time_min"
        }
    ],
    "fixed": {          # Propriétés de jeux de donnée fixée.
        "cond": "32"
    }
},
{                      # Un second graphique à deux axes.
    "name": "cond",
    "axis": [
        {
            "name": "cond (en log10)",
            "value": "cond"
        },
        {
            "name": "Temps",
            "value": "global_run_kernel_time_min"
        }
    ],
    "fixed": {
        "size": "1048576"
    }
},
{                      # Graphique en 3D.
    "name": "ratio",
    "axis": [
        {
            "name": "n",
            "value": "size"
        },
        {
            "name": "cond (en log10)",
            "value": "cond"
        },
        {
            # Axe z.
            "name": "temps",
            "value": "global_run_kernel_time_min"
        }
    ]
}
]
}

```

Propriétés Vous pouvez renseigner ici les propriétés des jeux de données à considérer. Si un point a été calculé à partir d'un jeu de données ne présentant pas les propriétés que vous demandez, celui-ci est ignoré. Chaque propriété peut être fixée avec une valeur.

Graphiques Les graphiques que nous voulons faire sont donnés dans une liste. Chaque graphique a un nom et des axes. Ne donnez pas le même nom à deux graphiques différents, car les noms de fichiers de données générés correspondant à chacun se basent dessus.

Axes Les axes sont indiqués dans des listes. Cette idée m'a été inspirée par le mode de fonctionnement du module `ctuning.space`, et me permet notamment de gérer les axes de façon beaucoup plus simple. Vous pouvez indiquer autant d'axes que vous voudrez, mais je ne garantis les résultats

que pour deux ou trois axes (mais pas à 100% tout de même).

En plus des propriétés des jeux de données, vous pouvez utiliser ces valeurs pour les axes :

- `global_build_time_max`,
- `global_build_time_min`,
- `global_code_md5sum`,
- `global_code_size`,
- `global_run_kernel_time_max`,
- `global_run_kernel_time_min`,
- `global_cpi`,
- `global_instructions_retired`,
- `cycles` (multiplication de `global_cpi` et `global_instructions_retired`).

Action Le module comporte 3 actions :

- **run** - Les graphiques demandés sont faits pour chaque espace à tracer.
- **run2** - Les graphiques demandés incluent tous les espaces.
- **run3** - Comme l'action précédente, mais avec les valeurs minimales. Ces dernières sont déterminées sur le dernier axe spécifié dans la description.

Chaque action sauvegarde ses résultats dans un dossier portant son nom, et dans le dossier spécifié par la clé `dir_output`, il faut appeler Gnuplot sur les scripts prévus pour ce programme afin de produire les graphiques.

4 Exemples

Des exemples d'utilisation de cMind se trouvent dans le dossier `scripts` à la racine de l'outil. Je ne saurais que trop vous conseiller de les essayer un peu tous, afin de voir les possibilités de cMind dont je n'ai pas parlé. Néanmoins, je vais vous introduire quelques exemples que j'ai regardé.

4.1 Visualisation

J'ai utilisé un module personnel pour les graphiques, voyons maintenant celui que cMind inclue, `ctuning.space`. Utilisons un fichier JSON pour utiliser cette fonctionnalité.

```
{
  "cm_action": "mpl2_2d_scatter_chart_a",      # Type de graphique.
  "ct_module_uoa": "ctuning.space",            # Dossier dans lequel se trouve les
  spaces.
  "ct_data_selection": ["acc_sum", "fast_acc_sum"], # Espace(s) à tracer.
  "use_multiple_graphs": "on",                # Permet d'utiliser différentes couleurs si plusieurs
  espaces sont précisés.
  "ct_dimensions": [                          # Axes.
    "##pipeline_output#state#cm_characteristics#global_code_size",
    "##pipeline_output#state#cm_characteristics#
      global_run_kernel_time_min"
  ]
}
```

Pour lancer le tracé, il faut taper

```
$ cm ctuning.space @fichier.json
```

Remarquez que cette fois-ci, aucune action n'est précisée sur la ligne de commande. C'est normal, car la clé `cm_action` permet de préciser l'action que nous souhaitons effectuer dans un fichier JSON, c'est une autre façon de faire mais nous aurions pu procéder comme à l'accoutumée. En ce qui concerne les axes, vous remarquerez sûrement que leurs valeurs ne vous sont pas étrangères, puisque les deux utilisées peuvent l'être dans le module que j'ai écrit. Le fait de passer par `ctuning.space` vous permet d'afficher de valeurs que je ne le permets avec mon module, mais pour savoir lesquelles utiliser, il vous faudra ouvrir le fichier `data.json` de l'espace à tracer, et galérer un peu.

4.2 Lister les options explorées

Pour voir les options explorées d'un espace, il suffit de taper

```
$ cm ctuning.scenario.compiler.optimizations view_ctuning_space  
view_ctuning_space_uoa=nom_espace
```

Vous pouvez rajouter le paramètre `output_known_characteristics=yes` pour avoir également les informations sur les temps d'exécution, la taille du code, etc...

4.3 Reproductibilité

La reproductibilité vous permet de refaire une exécution particulière d'un programme avec des combinaisons d'options de compilation déjà explorées. Pour cela, tapez

```
$ cm ctuning.pipeline.build_and_run_program reproduce  
reproduce_module_uoa=ctuning.space reproduce_data_uoa=nom_espace
```

Vous pouvez rajouter le paramètre `point=0` pour explorer un point en particulier (donc une combinaison). Ce sera 0 par défaut, donc le premier point. Les données concernant le point sont mises à jour après l'exécution et les mesures faites dessus.

4.4 Création de modules externes

Les modules externes étant une fonctionnalité assez importante dans l'utilisation de cMind, je pense que vous expliquer comment en créer vous sera utile.

4.4.1 Eléments de base

Le nom d'un module externe commence toujours par `"cme."`. Ses premières lignes sont également toujours les mêmes. Bien qu'elles ne soient pas obligatoires, le concepteur nous conseille de les mettre. Je rappelle que le langage à utiliser est Python, donc je parlerais de dictionnaires et de listes par la suite plutôt que de tableaux associatifs et indexés.

```
ini = {}  
cm_kernel = None
```

C'est une sécurité, je présume. Ensuite viennent les fonctions, chacune représente une action du module, elles doivent avoir obligatoirement un paramètre, communément appelé `i`, comme "informations" je suppose. Elle doit également retourner un dictionnaire, qui sera toujours le même dans le cas où tout va bien.

```
def mon_action(i):  
    print "Hello world !"  
  
    return {'cm_return' : 0}
```

Maintenant si vous devez renvoyer une erreur, il faudra un peu modifier le retour.

```
return {'cm_return': 1, 'cm_error': "Il y a une erreur"}
```

Ainsi, cMind se chargera d'afficher l'erreur lui-même en signalant que c'en est une. Appellons ce module `cme.perso.py` et mettons donc ce début de code dedans. Pour l'exécuter, il suffira de faire

```
$ cm cme.perso mon_action
```

Sans surprise, la console affiche `Hello world !`. Le paramètre `i` est très important, car il contient toutes les données que l'on transmettra au module, que ce soit en ligne de commande ou via un fichier JSON. Si une fonction n'a pas de paramètre, ou plus de un, cMind ne la considère pas comme une action du module. Pour voir ce que contient `i`, vous pouvez ajouter

```
print i;
```

dans la fonction. Vous obtiendrez dans la console le nom du module et l'action utilisée dans ce dictionnaire, ainsi que le format de sortie.

A présent, rajoutez n'importe quoi comme paramètre sur la ligne de commande, et vous les verrez apparaître dans `i`, n'oubliez pas que cela doit être de la forme `cle=valeur`. Cela fonctionne de la

même manière avec les fichiers JSON, toutes les données du fichier spécifié en paramètre se retrouvent dans `i...` `cMind` s'est chargé lui-même de récupérer le fichier, le transformer en dictionnaire, rajouter quelques informations au passage, pour ensuite appeler la fonction en lui donnant ce dictionnaire en paramètre.

4.4.2 Accès aux données

En console, pour récupérer les données relatives à des codes source, un compilateur ou tout autre chose, il faut taper

```
$ cm nom_module load cm_data_uoa=nom_donnee
```

Cela charge les données, et les affiche dans la console, ce qui ne nous avantage pas, car la console n'affiche que des chaînes de caractères, alors que les dictionnaires en Python sont très maniables. Mais `cm_kernel` définit une fonction qui nous donne l'accès aux données au sein de l'outil. Cette fonction est `cm_kernel.access` et demande un dictionnaire en entrée. Elle renvoie ensuite les données demandées, ou une erreur. Imaginons que je demande les informations concernant les codes sources des algorithmes de sommation, je dois faire

```
def mon_action(i):
    ii = {
        'cm_run_module_uoa': 'code.source',
        'cm_action': 'load',
        'cm_data_uoa': 'sum'
    }

    r = cm_kernel.access(ii)
    if (r['cm_return'] > 0): return r

    return {'cm_return': 0}
```

On reconnaît presque la ligne de commande en console. Si vous voulez vérifier que `r` contient les bonnes données, je vous conseille de passer par le format JSON, car vous aurez du mal à lire ce qui sera affiché dans la console. Pour ce faire, rajoutez :

```
import json

def mon_action(i):
    # ...

    s = json.dumps(r, indent=2)
    print s;

    # ...
```

On retrouve toutes les informations que l'on a données lors de la configuration des codes source, plus quelques autres gérées par `cMind`. Le test sur `r['cm_return']` est important car une erreur a été rencontrée si cette valeur est plus grande que 0.

4.4.3 Affichage

`cm_kernel` définit une autre fonction que l'on peut utiliser, s'apparentant à `print`, elle se nomme `cm_kernel.print_for_con` et s'utilise donc pour afficher des messages dans la console. Pourquoi utiliser cette fonction et non `print` tout simplement ? Parce que celle de `cMind` peut être désactivée. Cela est très pratique pour ne pas submerger l'utilisateur avec toutes les informations que les modules appelés affichent. Pour désactiver l'affichage de la fonction `cm_kernel.print_for_con`, utilisez `cm_kernel.set_print_mode`.

```
cm_kernel.set_print_mode({'mode': "off"});
cm_kernel.print_for_con("Je ne m'afficherais pas");
cm_kernel.set_print_mode({'mode': "on"});
```

4.4.4 Comportement par défaut

Pour finir, si vous ne renseignez pas d'action à exécuter par le module, alors celui-ci appellera la fonction qui se nomme `default`. Si elle n'est pas définie, eh bien Python plantera. Vous pouvez toujours la rajouter pour afficher un message plus propre qu'un plantage de Python.

4.4.5 Résumé

Voici maintenant le module que j'ai décrit dans son intégralité, agrémenté de la fonction `default` :

```
ini = {}
cm_kernel = None

import json

def default(i):
    cm_kernel.print_for_con("Rien à faire")
    return {'cm_return': 0}

def mon_action(i):
    s = json.dumps(i, indent=2)          # Pour que i soit utilisé dans le module.
    cm_kernel.print_for_con(s)

    ii = {
        'cm_run_module_uoa': 'code.source',
        'cm_action': 'load',
        'cm_data_uoa': 'sum'
    }

    r = cm_kernel.access(ii)
    if (r['cm_return'] > 0): return r

    s = json.dumps(r, indent=2)
    cm_kernel.print_for_con(s)

    return {'cm_return': 0}
```

Voilà, vous devriez pouvoir écrire vos propres modules maintenant, mais il faudra suivre les exemples du concepteur assez souvent pour faire des choses intéressantes.

4.4.6 Noms de variables

Si vous êtes amené à lire les codes sources du concepteur, vous remarquerez très vite une chose : les noms de variables ne doivent pas être explicites dans cMind. Ils doivent être juste les initiales des données que les variables représentent. En plus de rendre l'écriture rapide au début, et plus difficile à maintenir par la suite, cela découragera vite vos collègues de s'attaquer à la lecture de vos modules. Ainsi, vous en resterez le seul maître.

5 Graphiques

Voici maintenant les graphiques que j'ai obtenus à partir des mesures faites par cMind. Je compare les combinaisons d'options à l'option `-O3` seule afin de voir si cMind a réussi à trouver une (ou des) combinaison(s) plus intéressante(s) que `-O3` pour les algorithmes.

Intel(R) Core(TM) 2 Duo CPU P8800 @ 2.66GHz, x86_64
Linux version 3.2.0-3-amd64 (Debian 3.2.23-1)
gcc-4.6

FIGURE 1 – Environnement pour les mesures

Type 1.a Nombre de cycles vs taille de vecteur. Contionnement fixé à 10^8 .

Type 1.b Nombre de cycles vs taille de vecteur. Contionnement fixé à 10^{32} .

Type 2 Nombre de cycles vs conditionnement. Taille de vecteur fixée à 2^{20} .

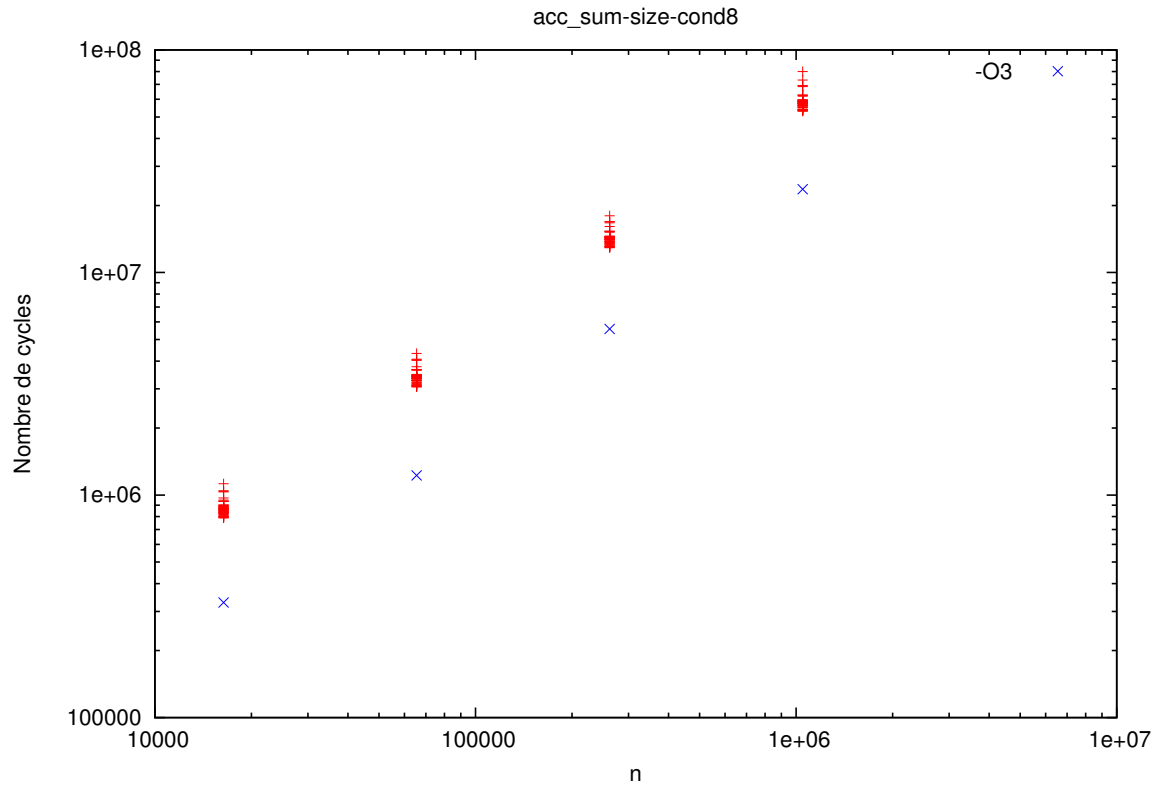


FIGURE 2 – Type 1.a - AccSum

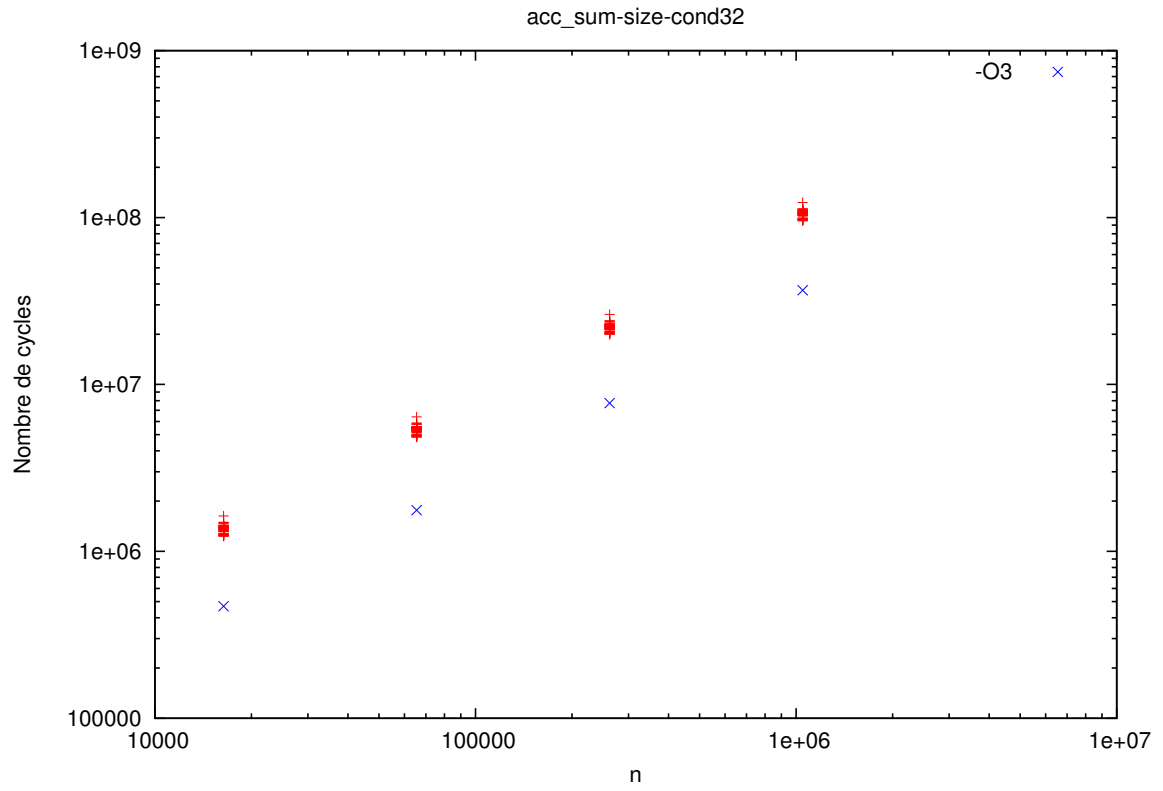


FIGURE 3 – Type 1.b - AccSum

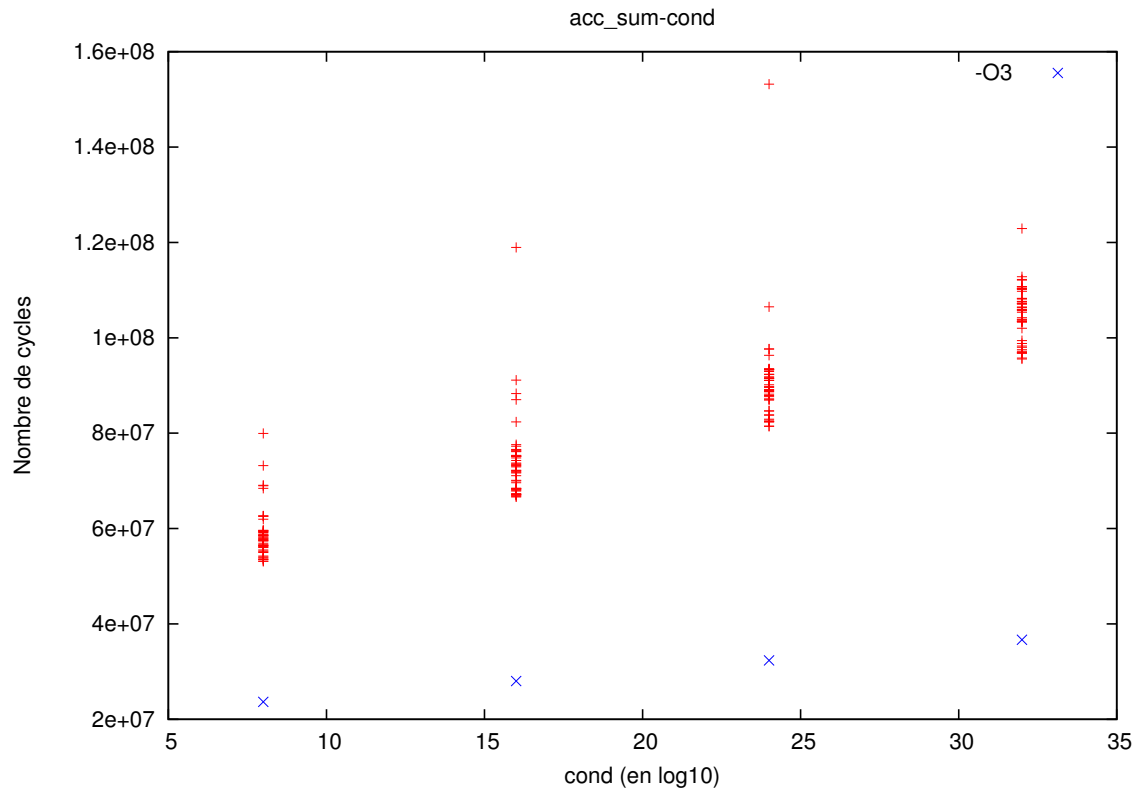


FIGURE 4 – Type 2 - AccSum

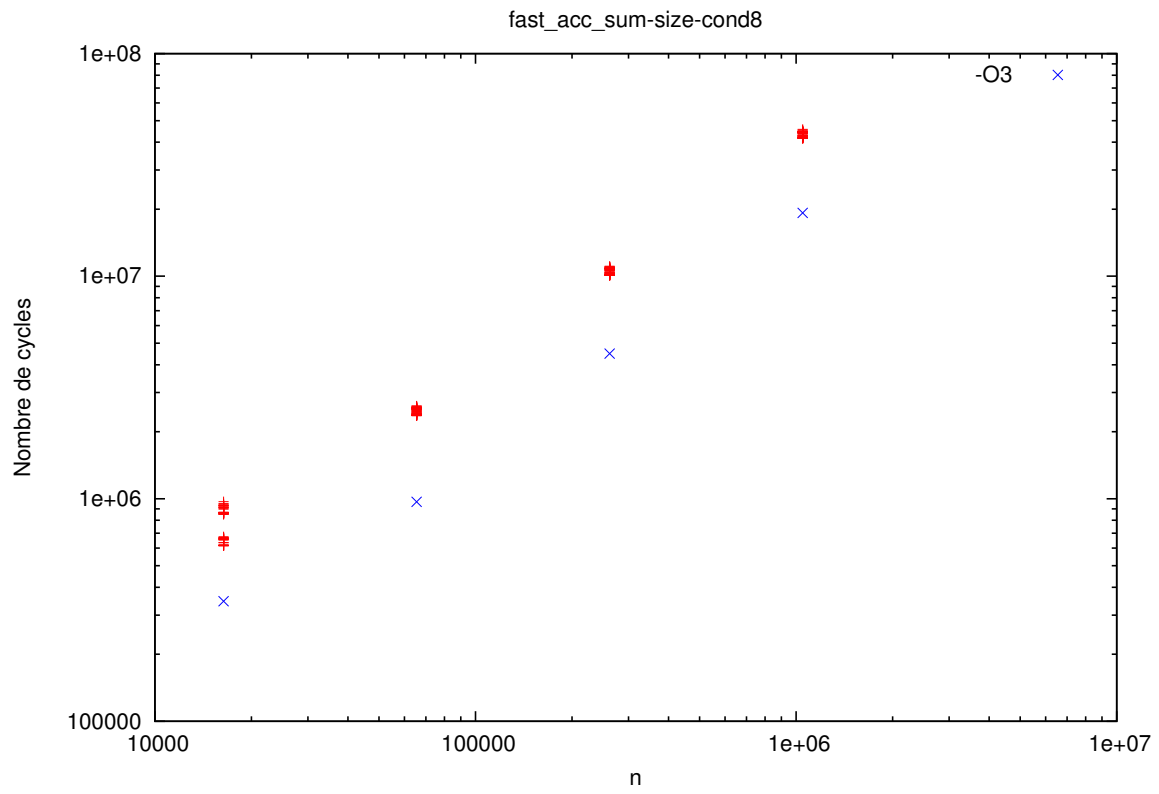


FIGURE 5 – Type 1.a - FastAccSum

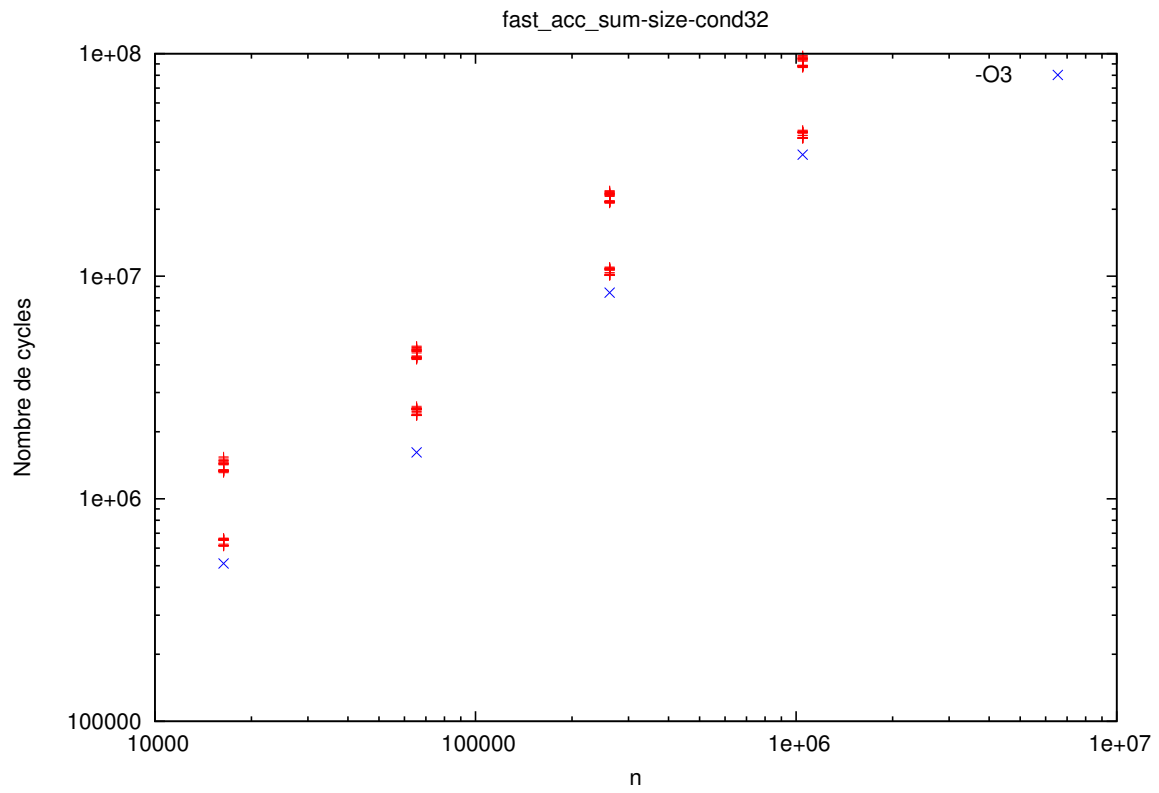


FIGURE 6 – Type 1.b - FastAccSum

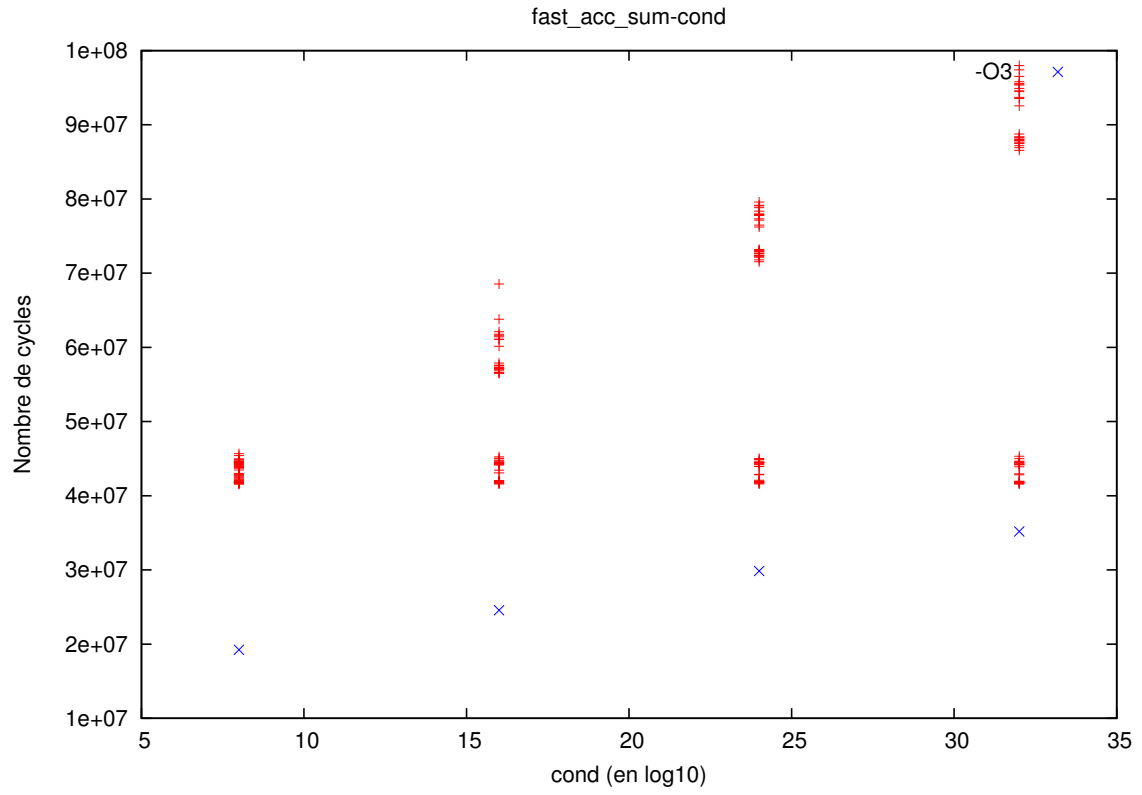


FIGURE 7 – Type 2 - FastAccSum

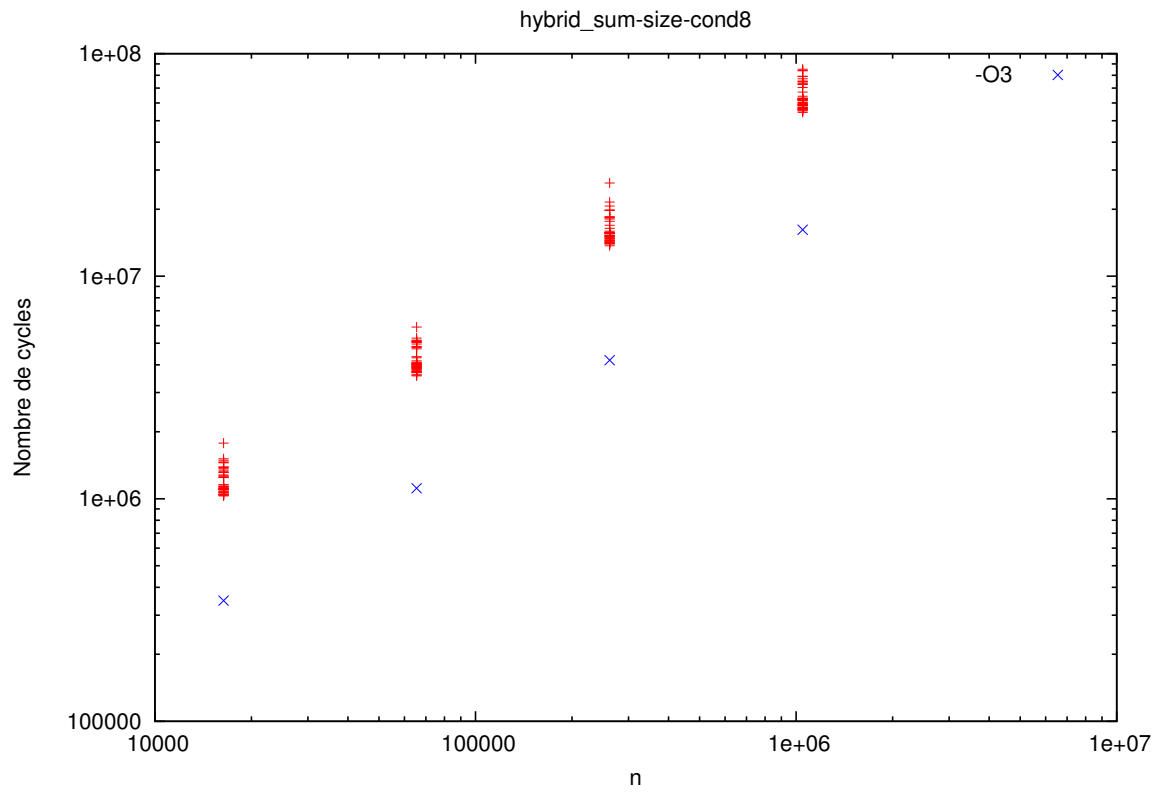


FIGURE 8 – Type 1.a - HybridSum

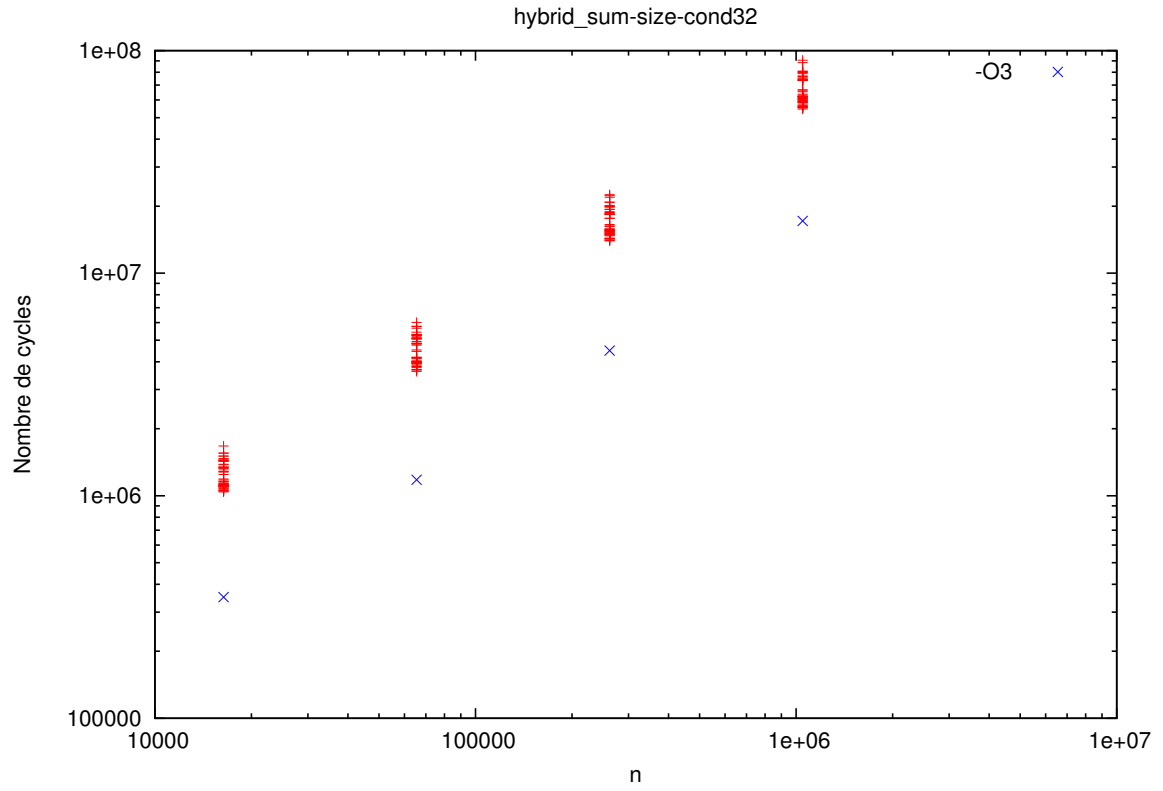


FIGURE 9 – Type 1.b - HybridSum

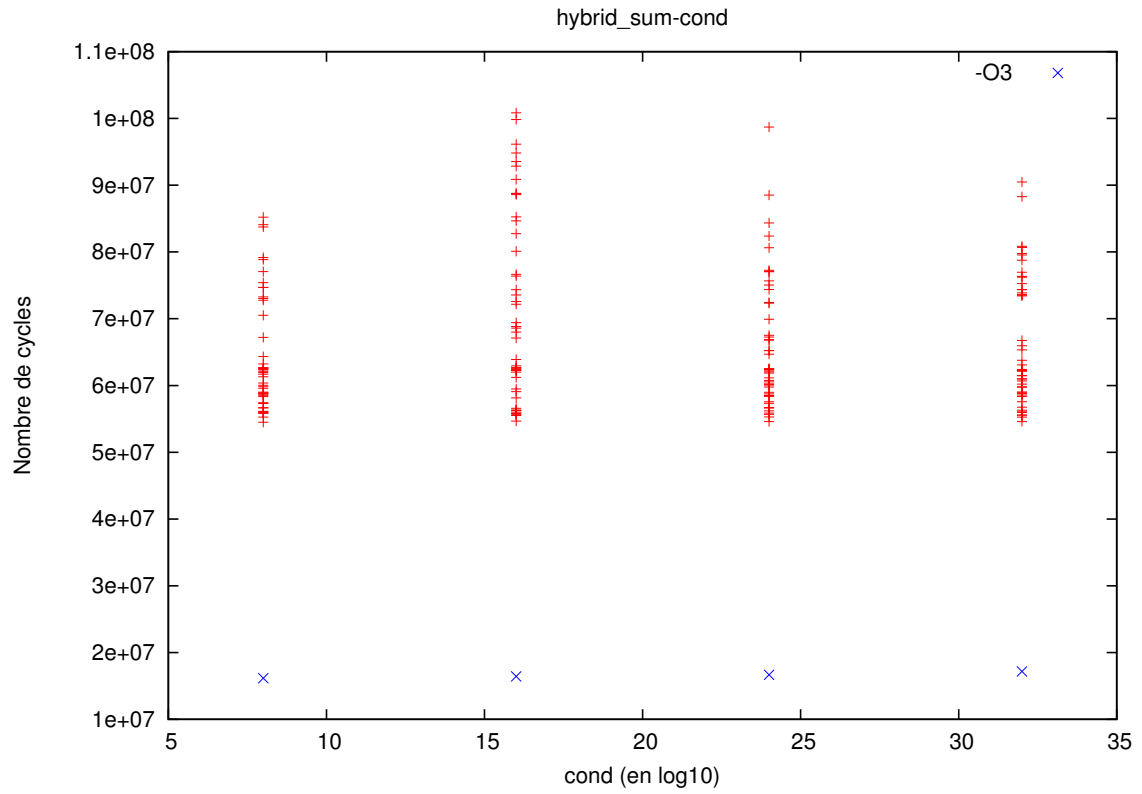


FIGURE 10 – Type 2 - HybridSum

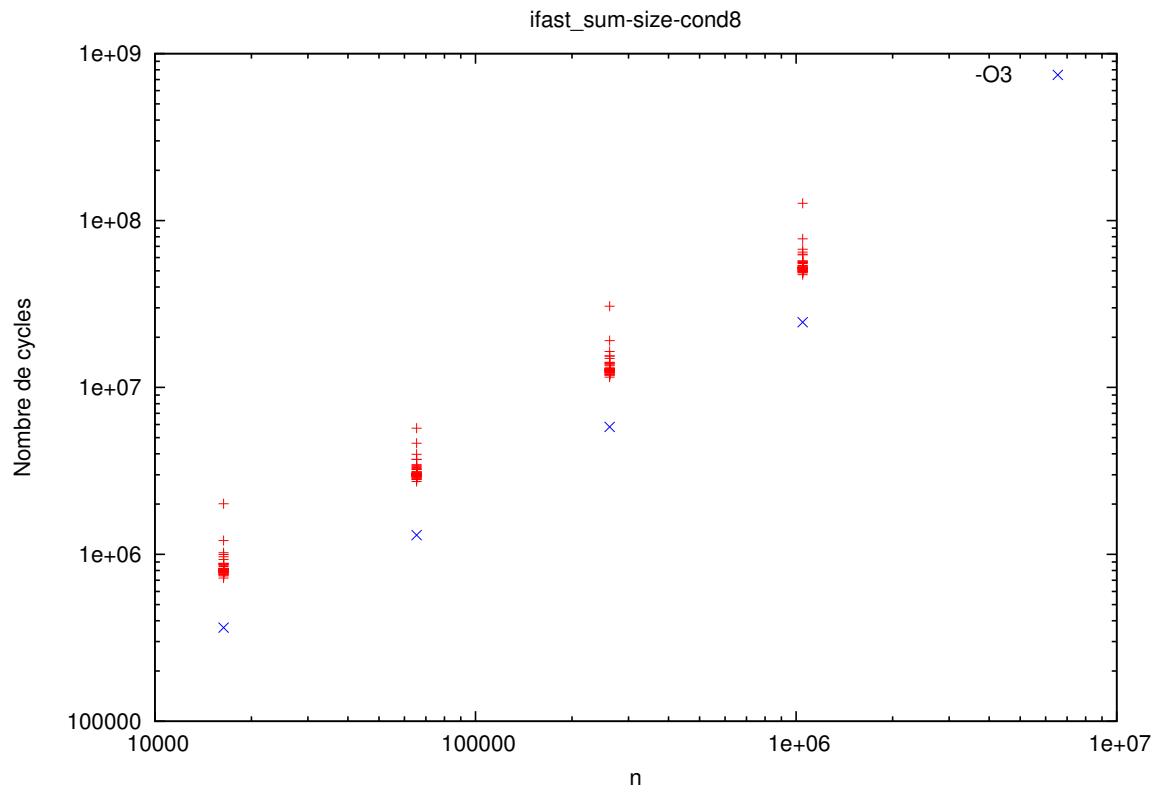


FIGURE 11 – Type 1.a - iFastSum

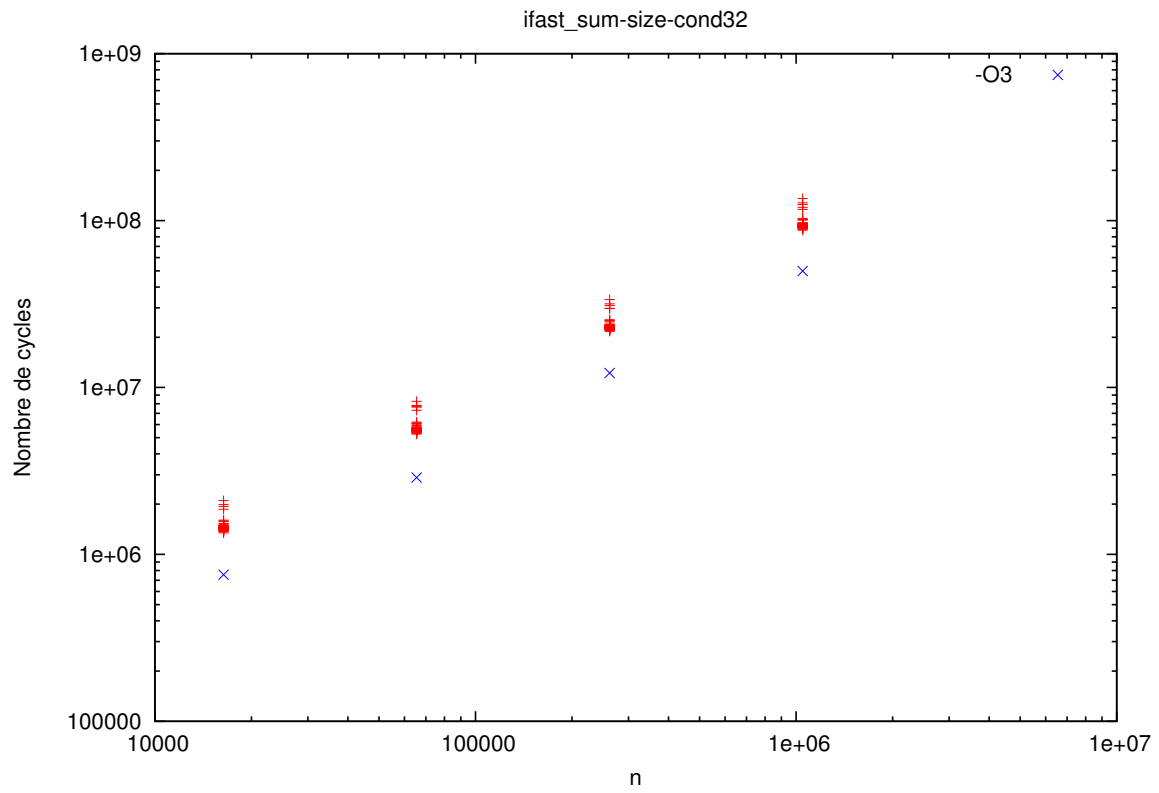


FIGURE 12 – Type 1.b - iFastSum

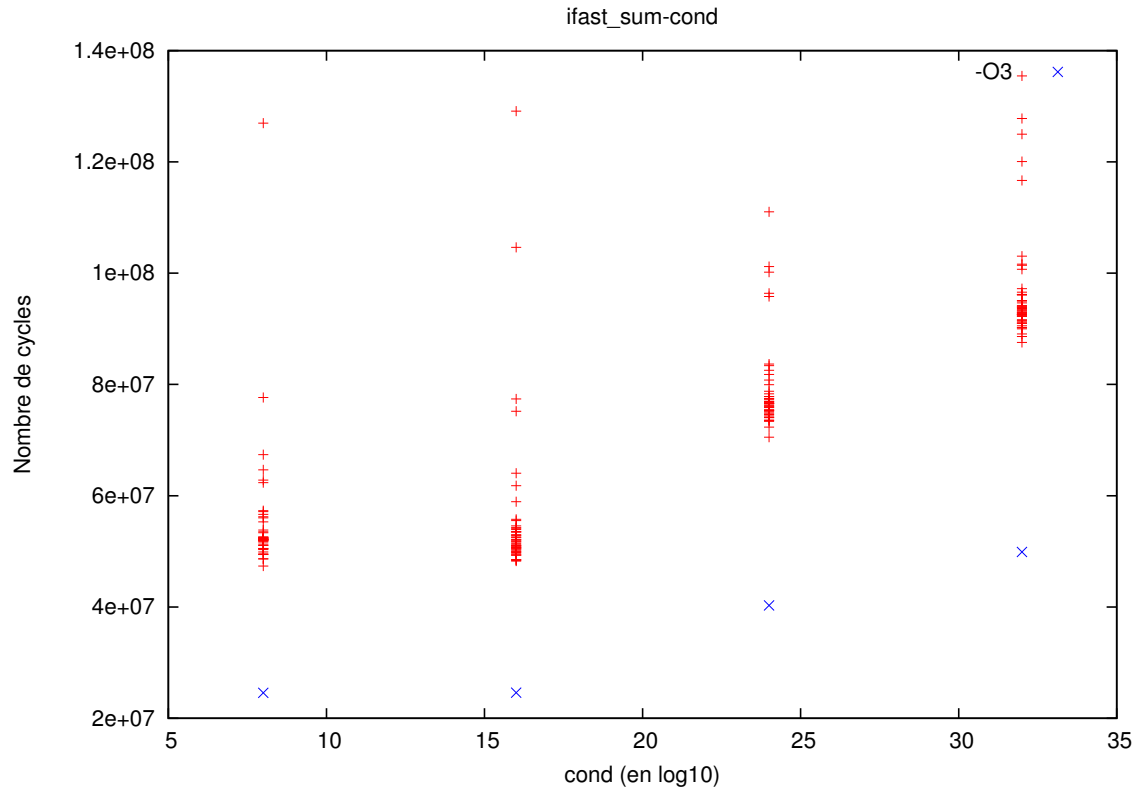


FIGURE 13 – Type 2 - iFastSum

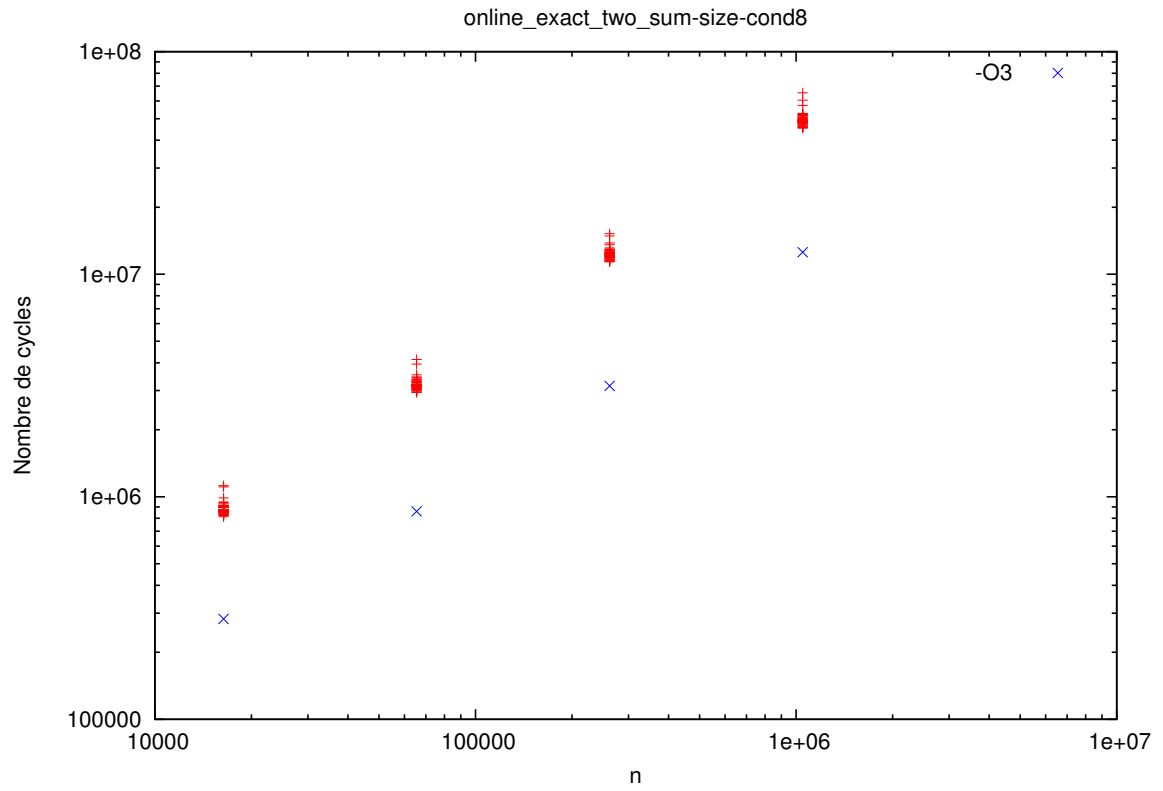


FIGURE 14 – Type 1.a - OnlineExact

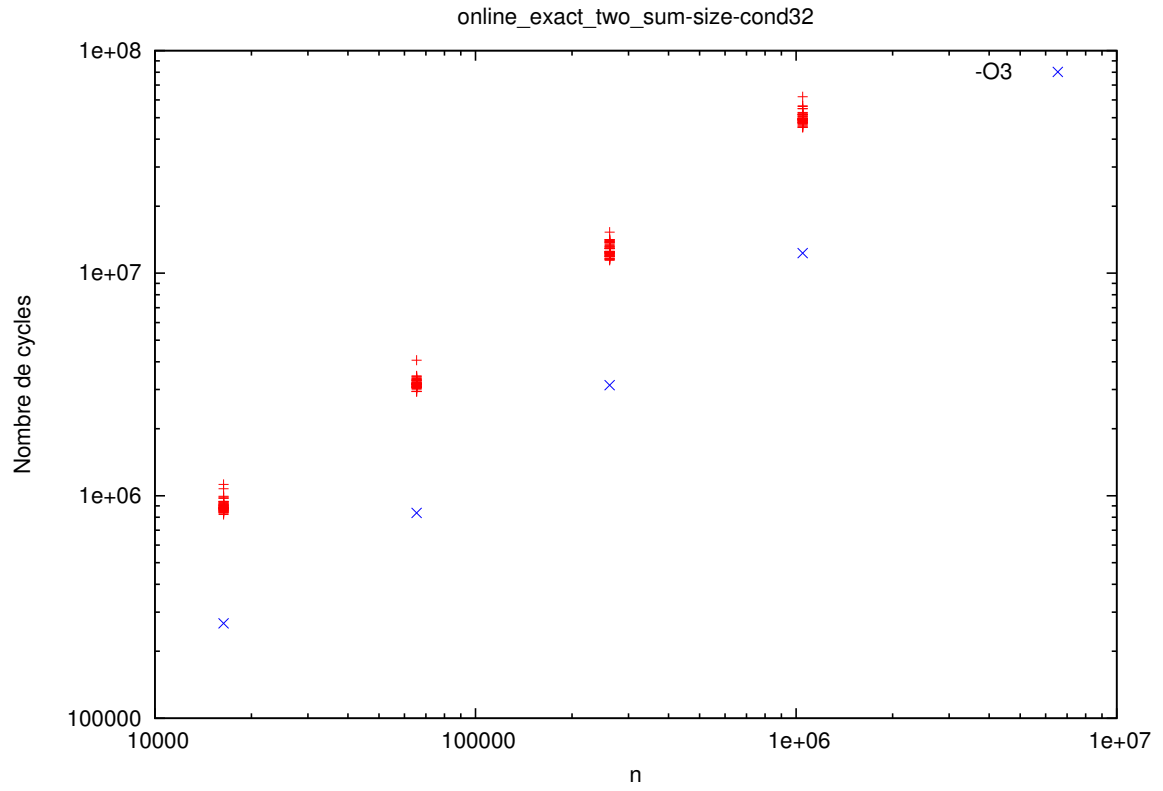


FIGURE 15 – Type 1.b - OnlineExact

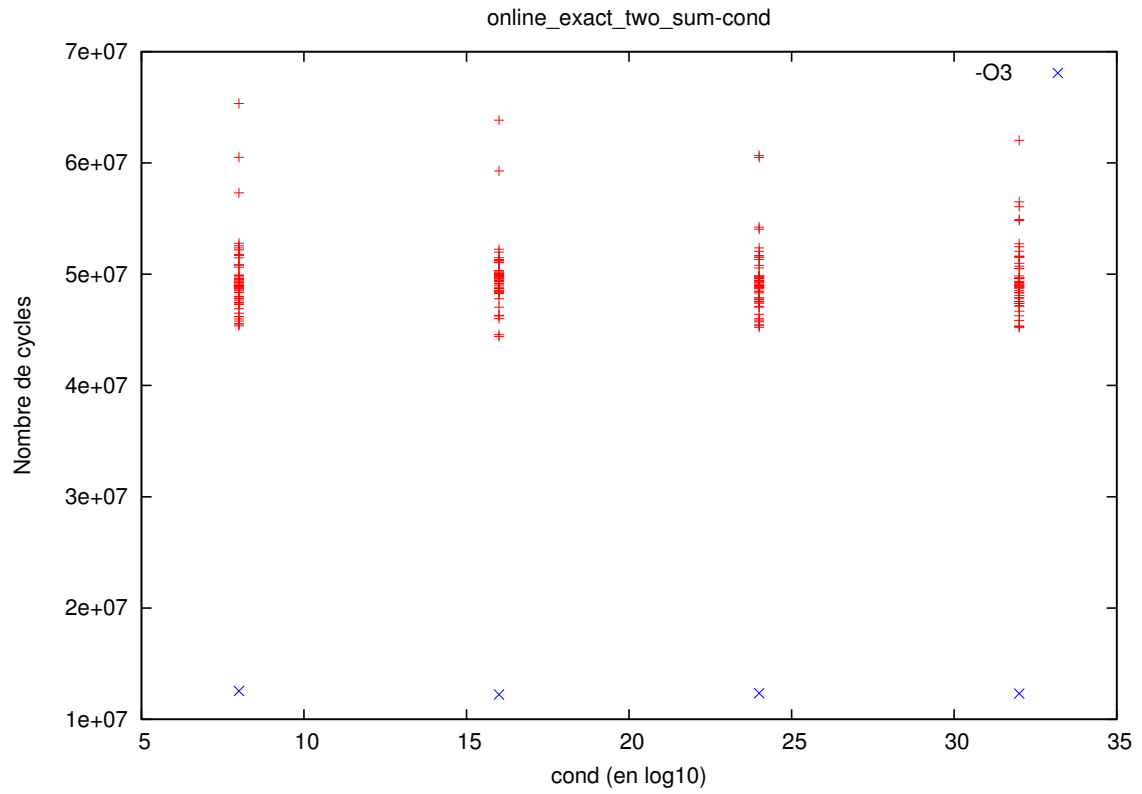


FIGURE 16 – Type 2 - OnlineExact

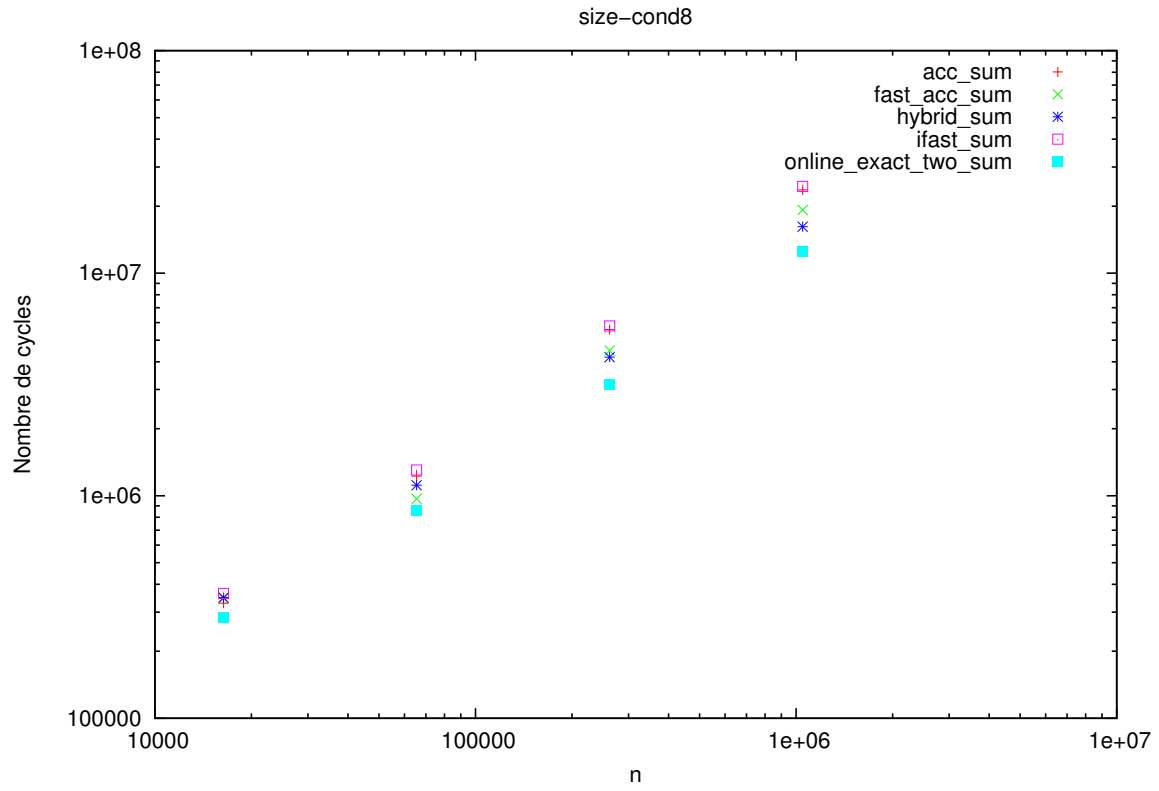


FIGURE 17 – Type 1.a - Tous (valeurs minimales)

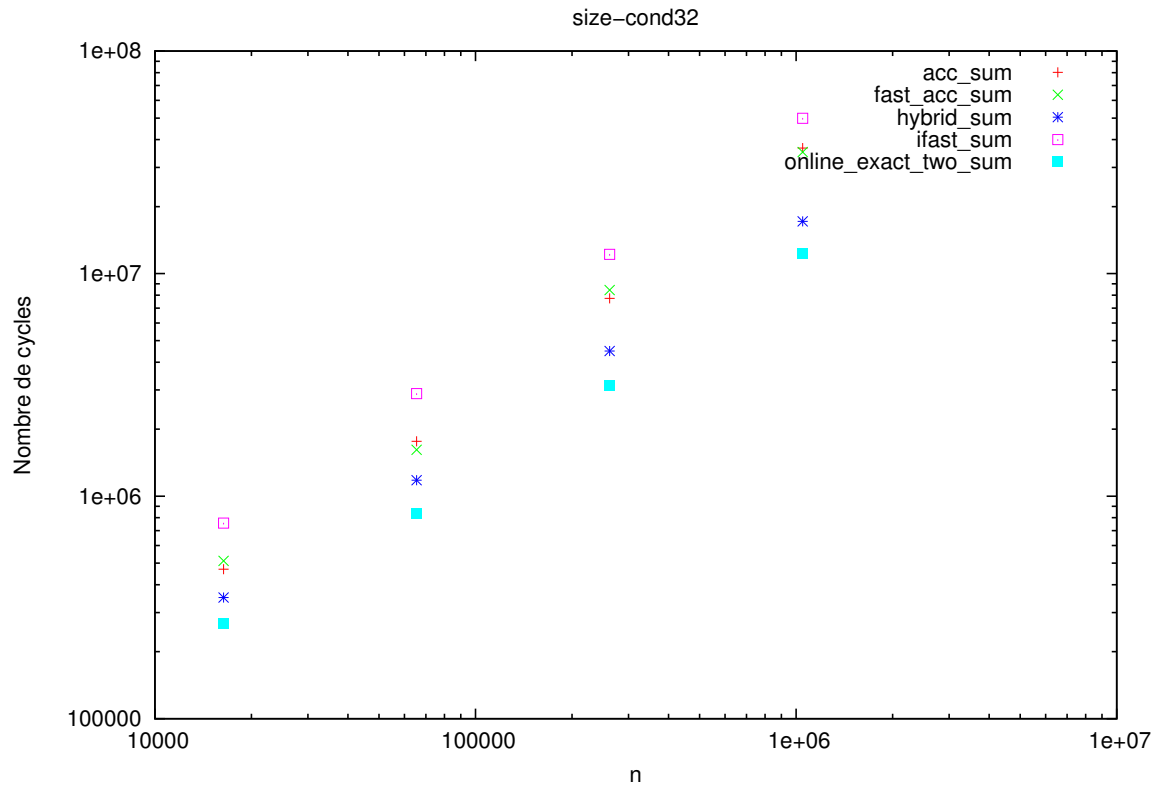


FIGURE 18 – Type 1.b - Tous (valeurs minimales)

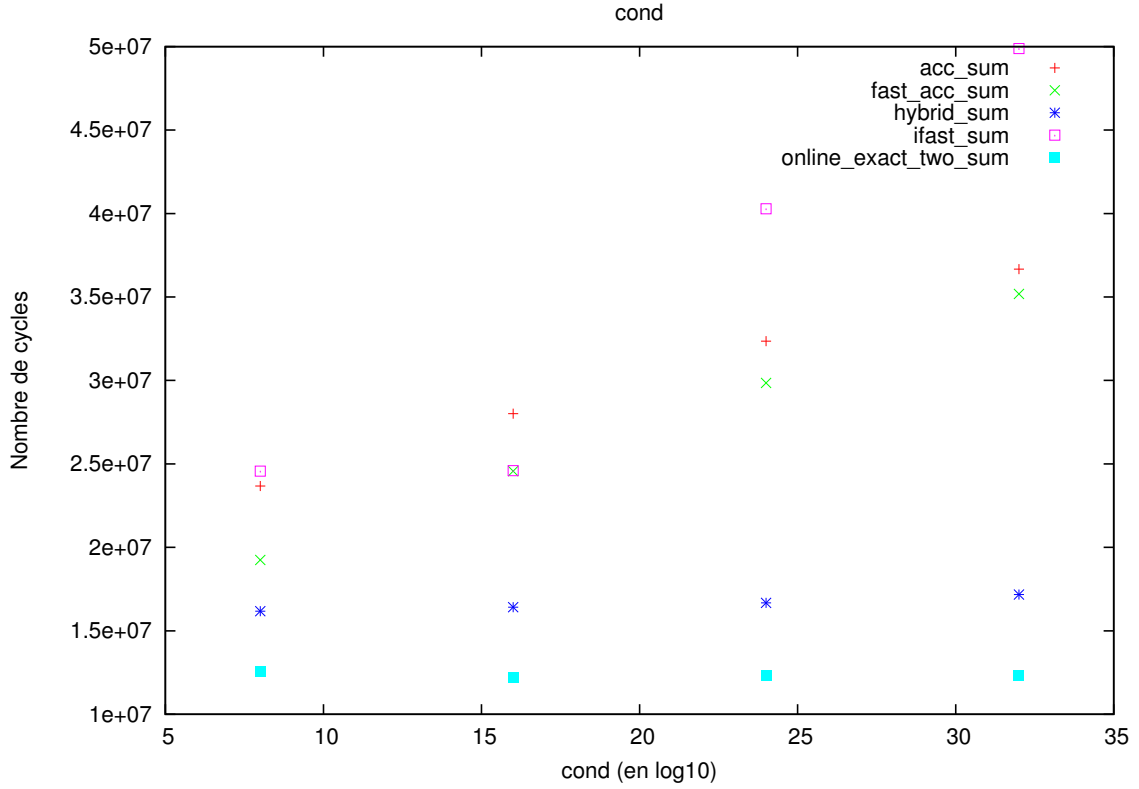


FIGURE 19 – Type 2 - Tous (valeurs minimales)

5.1 Commentaires

On peut dire que cela est assez décevant comme résultats. 50 combinaisons testées semblait être suffisant pour en trouver une qui battrait -O3, or ce n'est pas le cas. Il faudrait maintenant affiner la sélection des options, seulement l'écriture d'un nouveau module externe serait obligatoire. Une idée serait de lister pour chaque option lesquelles n'agissent pas en synergie avec et ainsi mettre de côté au fur et à mesure les "mauvaises" options. Cela pose un problème tout de même, car il serait bien difficile de déterminer les synergies entre options dans l'absolu, car j'imagine que celles-ci sont différentes en fonction des problèmes. De plus, certaines options prennent différentes valeurs, parfois des nombres, donc il serait encore plus compliqué de savoir quelles valeurs seraient intéressantes ou pas. Le machine learning pourra peut-être répondre à ce problème, le concepteur de cMind travaille là-dessus mais ce n'est pas encore au point.

A Likwid

Vous aurez remarqué que les mesures sont exprimées en nombres de cycles et non en temps sur les graphiques, alors que vous n'obtenez que des temps en utilisant cMind. C'est parce que ce dernier peut utiliser un autre outil, Likwid[3], qui permet de mesurer le nombre de cycles.

A.1 Présentation

A.2 Installation

Pour installer Likwid correctement, il vous faudra être super-utilisateur. Après l'installation, vous pourrez l'utiliser sans l'être. Allez dans le dossier `./misc`, et lancez le script `install_likwid.sh`. Comme c'est un script à part, les variables d'environnement permettant de localiser cMind sont redéfinies dedans, pensez à les modifier si besoin. Le script se charge d'installer Likwid et de le configurer. Likwid se sert des compteurs matériels, il doit avoir accès aux fichiers `/dev/cpu/*/msr`. Il faudra déjà activer les compteurs via la commande

```
$ sudo modprobe msr
```

La façon dont je configure Likwid permet de l'utiliser sans être super-utilisateur car il utilisera un daemon appartenant à ce dernier, avec un `setuid`. Ceci n'est pas la meilleure solution, mais la plus simple.

Likwid est configuré pour cMind, mais ne l'est pas pour votre machine, il vous faudra donc le télécharger et l'installer comme indiqué sur le site si vous souhaitez vous en servir autrement que via cMind.

A.3 Utilisation

Je vais parler seulement de `likwid-perfctr`, car c'est le programme qui mesure les nombres de cycles, mais aussi des groupes d'événements matériels, avec lesquels je ne suis pas très au point. Donc j'explique le fonctionnement général, mais je ne pourrais pas entrer dans les détails.

Pour lister les groupes de performances, faites (en mode super-utilisateur)

```
# likwid-perfctr -a
```

et pour les événements

```
# likwid-perfctr -e
```

Pour préciser le groupe que nous souhaitons utiliser, il faut utiliser l'option `-g`. L'option `-o` permet de sauvegarder les résultats des mesures dans un fichier (au format `.txt`). L'option `-C` permet de préciser sur quel processeur nous voulons faire les mesures, il est possible d'en donner plusieurs, tel `-C 0-3` pour faire sur les processeurs 0 à 3 ou `-C 1, 3` pour demander seulement sur les processeurs 1 et 3. L'option `-m` permet d'utiliser des marqueurs, qui serviront à faire des mesures seulement dans une portion du code que nous indiquerons, bien évidemment. Pour cela, il faudra rajouter quelques lignes que voici.

```
likwid_markerInit();           // Initialisation des marqueurs.
likwid_markerClose();          // Libération des marqueurs (ou fermeture).
likwid_markerStartRegion(const char *str); // Début d'une région (portion) à
    considérer.
likwid_markerStopRegion(const char *str);  // Fin d'une région.
```

Vous aurez compris qu'une région est déterminée par son nom sous la forme d'une chaîne de caractères. Il faut donc spécifier la même chaîne en début et en fin de région à mesurer.

A.4 Fiabilité

Un outil de mesures c'est bien, mais quid de sa fiabilité ? Pour cela, j'ai testé personnellement l'outil sur les codes compilés sans cMind, pour faire quelques graphiques. Vous pouvez retrouver les scripts dans `./likwid_test`. Chaque exécution pour un algorithme et un jeu de données est effectuée 10 fois, je récupère la valeur minimale.

Intel(R) Core(TM) 2 Duo CPU P8800 @ 2.66GHz, x86_64
Linux version 3.2.0-3-amd64 (Debian 3.2.23-1)
gcc-4.6
gcc -O3

FIGURE 20 – Environnement pour les mesures

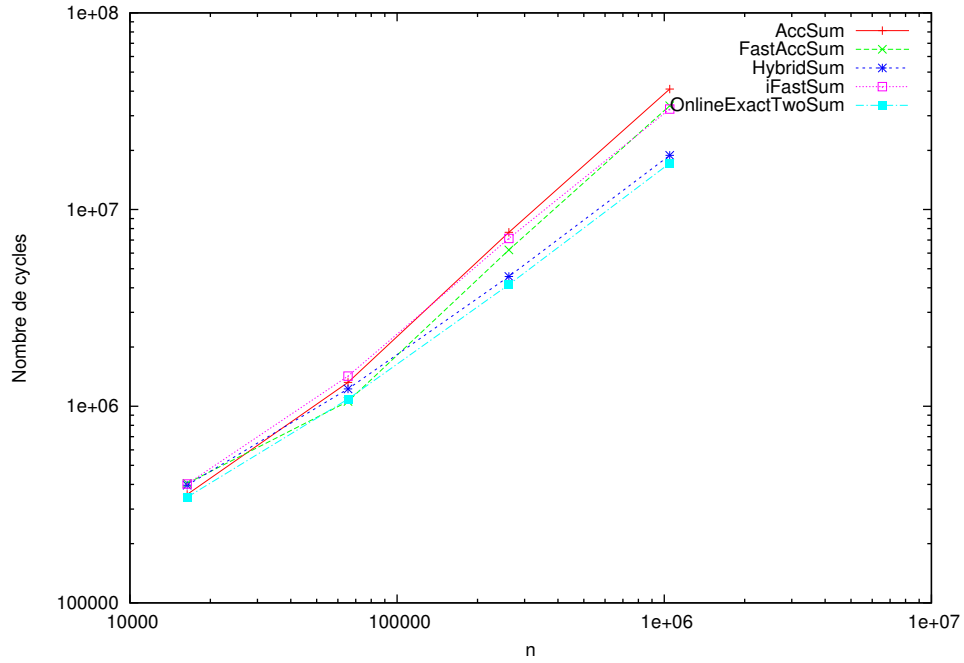


FIGURE 21 – Nombre de cycles vs taille du vecteur de conditionnement 10^8

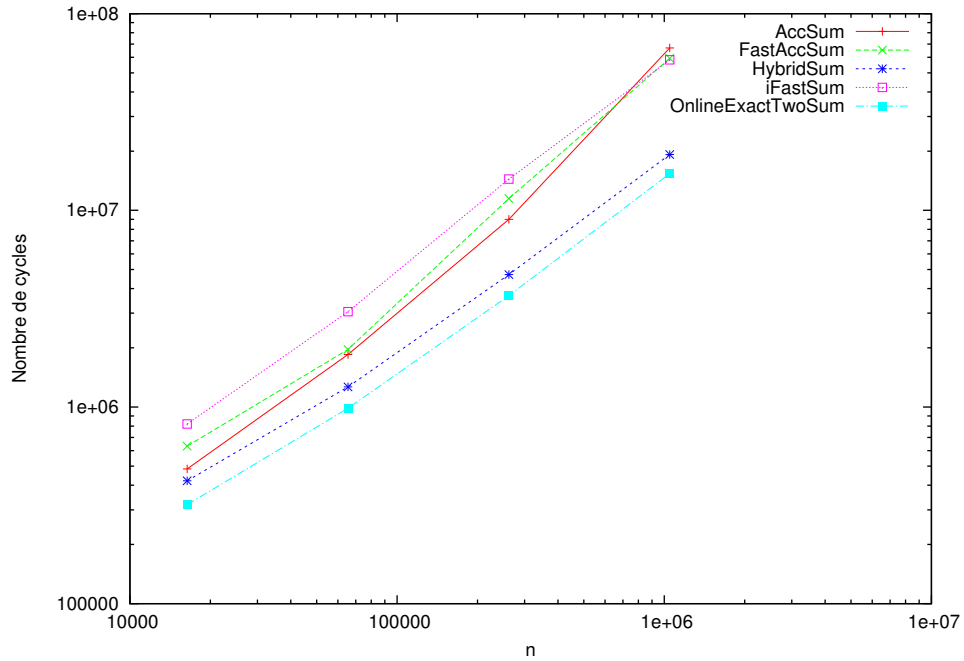


FIGURE 22 – Nombre de cycles vs taille du vecteur de conditionnement 10^{32}

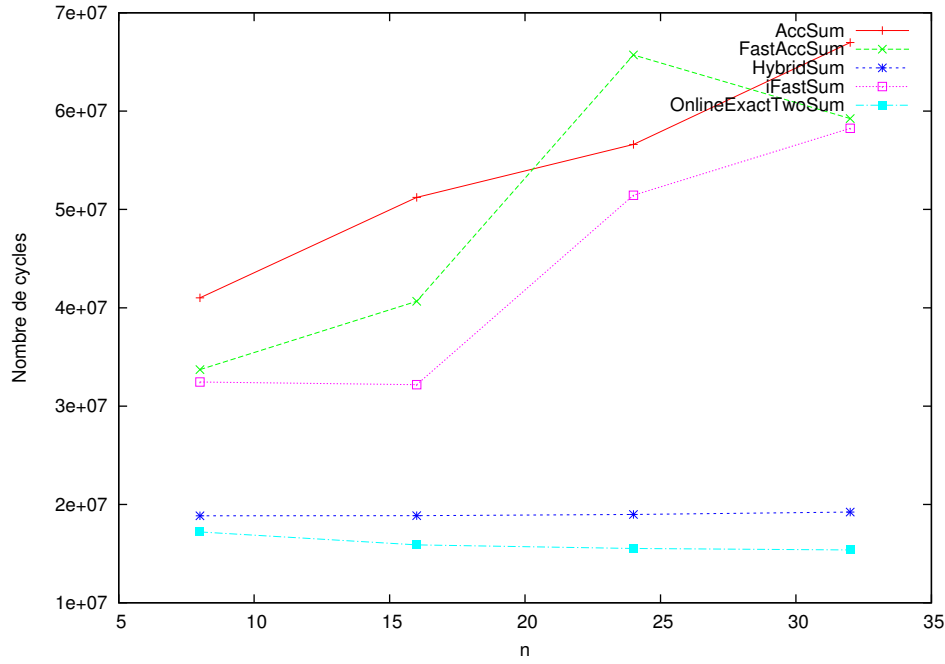


FIGURE 23 – Nombre de cycles vs conditionnement du vecteur de taille 2^{20}

Les résultats obtenus sont attendus, OnlineExact et HybridSum ne sont effectivement pas sensibles au conditionnement, et toutes les courbes en fonction de la taille augmentent de façon linéaire. Likwid peut être considéré comme fiable pour ces algorithmes.

A.5 Instrumentation avec cMind

L'utilisation est un peu différente avec cMind. En effet, ce dernier active l'option `-m` et je n'ai pas trouvé comment l'en empêcher, il est donc nécessaire de rajouter les marqueurs. L'instrumentation des codes avec Likwid nécessite des fichiers template, d'extension `.cm_template`, en plus de l'extension du fichier. Il faut également éditer le fichier `data.json` décrivant les codes sources, et rajouter la clé `source_files_templates` en lui fournissant la liste des fichiers sous forme de tableau (donc entre `[ ]`), mais sans l'extension `.cm_template`. Le fichier est à spécifier dans les fichiers sources également (`source_files`). cMind va donc rajouter les marqueurs pour nous, ainsi que les inclusions nécessaires, mais il va falloir lui dire où. Pour cela, il est nécessaire de rajouter des commentaires dans le fichier template, et cMind rajoutera les marqueurs et inclusions juste au-dessus.

```
/* CM_TEMPLATE_INCLUDE */           // #include <likwid.h>
/* CM_TEMPLATE_PROGRAM_START */      // likwid_markerInit();
/* CM_TEMPLATE_PROGRAM_END */        // likwid_markerClose();
/* CM_TEMPLATE_KERNEL_START */       // likwid_markerStartRegion(const char *str);
/* CM_TEMPLATE_KERNEL_END */         // likwid_markerStopRegion(const char *str);
```

Ainsi, cMind récupérera les CPI mesurés par Likwid et le nombre d'instructions exécutées. Nous pouvons donc facilement récupérer le nombre de cycles à partir de ces deux valeurs.

Si vous regardez le fichier `./sources/main.c.cm_template`, vous verrez les commentaires que j'y ai rajouté pour l'instrumentation de Likwid. Pour dire à cMind d'utiliser Likwid lors des explorations, il faut rajouter quelques informations dans le fichier de configuration.

```
"use_likwid": "no",                  # Utilisation.
"likwid_package_name": "likwid 3.0.0", # Version de Likwid.
"likwid_fuzzy_match": "yes",          # Complétion des noms.
"likwid_params": {
  "instrumentation": "yes",           # Instrumentation (nécessite les fichiers template).
  "use_sudo": "no",                  # Demande à être super-utilisateur pour l'accès aux
    compteurs.
  "processor_pinning": "0",           # Processeur(s) utilisé(s) (-C).
```



```

    "measure_performance_group": "DATA",      # Groupe d'évènements (-g).
    "measure_event_set": ""                  # Evènements.
},

```

Dans un premier temps, vous pouvez juste mettre à "yes" la valeur de la clé `use_likwid` du fichier `./json/explore_*.json`, mettez `calibrate_kernel` à "no" car nous n'avons plus besoin de tester plusieurs valeurs de `CT_REPEAT_MAIN` pour chaque combinaison, et tapez

```
$ cm code.source update @./json/code_template.json
```

Il faudra mettre le fichier `main.c.cm_template` dans le dossier du projet (sum), puis relancer les explorations, via `./launch -e`

B ICC

Il est possible d'installer ICC. Seulement, je ne connais pas ce compilateur et je ne me suis pas penché dessus. De plus, il faudra que celui-ci soit déjà installé sur votre machine, car nous ne ferons que renseigner à cMind comment l'utiliser. Pour cela, lancer le script `install_icc.sh` à partir du dossier `./misc`, modifiez les variables d'environnement définies au préalable comme il convient. Le script crée un package au nom de `icc-generic` puis l'installe grâce au module externe en fonction de votre machine. J'imagine que ICC et GCC se ressemblent beaucoup au niveau des options pour dire au compilateur où se trouvent les headers, le format des fichiers objets, etc... Cependant, si vous connaissez ICC, vous saurez comment le configurer. Avec de lancer le script, ouvrez les deux fichiers `./config/icc_package_*`. Là-dedans, vous pourrez préciser comment utiliser ce compilateur. Ces deux fichiers étant basés sur ceux de GCC, je vous laisse le soin de les compléter comme il se doit pour ICC, les noms des variables sont assez explicites. Pour demander à compiler avec ICC lors des explorations, remplacer `gcc` par `icc` à la clé `compiler_package_name` du fichier de configuration. Vous avez donc la possibilité d'utiliser votre compilateur favori en suivant la même procédure pour celui que vous voudrez utiliser. Néanmoins, le module d'installation (`cme.install.py`) n'est utilisable que pour GCC, ICC et Likwid car je ne l'ai écrit que pour ces trois outils.

C FAQ

C.1 Ajouter un nouvel algorithme

Ajouter un nouvel algorithme à mes codes sources ne sera pas compliqué, du moment que la fonction qui l'utilise respecte le prototype

```
double function(double *vector, unsigned int size);
```

Mettez votre fonction dans un fichier C, à la rigueur le `main.c` pour ne pas vous prendre la tête, et rajouter son nom au tableau de pointeurs de fonctions nommé `algos` dans le fichier `main.c`, dans le dossier `./sources`, évitez de toucher les fichiers de cMind le plus souvent possible. Ensuite, vous pouvez rajouter le nom de l'algorithme à l'énumération de ceux que j'utilise, pour rendre plus clair le programme. Chaque algorithme est appelé en fonction de la ligne de commande. Suivez la méthode de fonctionnement pour le test sur le premier argument de la ligne de commande, mais c'est à vous de définir quelle valeur permettra l'appel de votre fonction. Appelez la fonction `doStats` de la même manière que pour les algorithmes déjà en place, avec l'indice de votre fonction dans le tableau `algos`. Maintenant, il va falloir rajouter une nouvelle commande dans la clé `run_cmds` du fichier `data.json` décrivant les codes sources de ce programme. Rajoutez votre commande, avec un nom logique de préférence par rapport à l'algorithme, le point le plus important est la ligne `run_cmd_main` dans laquelle il va falloir mettre les paramètres passés à l'exécutable. N'oubliez pas que les jeux de données sont transmis automatiquement avec l'expression déjà vu plus haut.

Vous pouvez également utiliser mes codes sources pour faire travailler d'autres types d'algorithmes, mais je ne vous le conseille pas car vous perdriez du temps si vous ne réutilisez pas la fonction `doStats` qui marchent avec le tableau de pointeurs de fonctions.

C.2 Ajouter un jeu de données

Vous avez déjà vu comment ajouter un nouveau jeu de données.

```
$ cm dataset add cm_data_uoa=nom_jeu_de_donnees @description.json
```

Revoyez la section parlant des descriptions des entrées de cMind pour savoir comment remplir la description des jeu de données. Il vous faudra déplacer les données correspondantes dans le dossier créé par cMind contenant la description de la nouvelle entrée de jeu de données.

C.3 Faire un nouveau test (algorithme + jeux de données)

Pour tester vos algorithmes (ou programme) avec vos jeux de données, je vous conseille d'utiliser un nouveau fichier JSON avec les mêmes paramètres que ceux du fichier `./json/explore_*.json`, et de modifier les paramètres qui servent à préciser la commande, les codes sources, le jeu de données, l'espace de stockage, etc... puis de lancer

```
$ cm cme.explore_compiler_flags run @fichier.json
```

C.4 Ajouter une option de compilateur

La procédure d'ajout de compilateur est similaire aux autres entrées, nous avons déjà vu qu'il fallait fournir des informations relatives aux options de compilation pour les compilateurs. Cependant, si la suppression d'une option est aisée, son ajout l'est beaucoup moins. D'ailleurs, je ne me suis pas penché dessus, il vous faudra essayer tout seul. Observez le fichier `./json/compiler.json` pour essayer de comprendre, la principale difficulté réside dans le fait que chaque option est désigné par un UID unique. Mais comment le déterminer, telle est la question.

C.5 Compiler avec une bibliothèque externe

Il est très simple de compiler avec une bibliothèque externe, nous l'avons déjà vu. Lorsque vous définissez une nouvelle commande pour une entrée de codes sources, vous pouvez rajouter le paramètre de clé `build_set_env`, dont la valeur sera un tableau indexé. Vous pourrez redéfinir chaque variable d'environnement utilisé par le script de compilation que vous choisirez à l'exploration. Utilisez la clé `CM_LD_FLAGS_MISC` de préférence, celle-ci n'est pas utilisée lorsque l'on se sert de GCC.

```
"build_set_env": {  
  "CM_LD_FLAGS_MISC": "-lpapi" # Exemple.  
}
```

C.6 Faire une exploration sans jeu de données

Jusque là, nous avons toujours utilisé des jeux de données avec nos codes. Si jamais vous avez un programme à tester qui n'en demande pas, il faudra rajouter la valeur `"dataset.empty"` dans la liste de clé `dataset_classes_uoa` des commandes de codes sources.

```
{  
  "run_cmds" : [  
    "default" : {  
      "dataset_data_uoa" : [  
        "dataset.empty"  
      ],  
    },  
  ],  
}
```

Références

- [1] cTuning, Gregory Fursin, <http://ctuning.org/>
- [2] cMind, Gregory Fursin http://ctuning.org/cm/wiki/index.php?title=Main_Page
- [3] Likwid, <https://code.google.com/p/likwid/>